

01_cifar10_dataset_preparation_enhanced

April 9, 2025

1 CIFAR-10 Dataset für CNN-Autoerkennung

Dieses Notebook lädt den CIFAR-10 Datensatz und bereitet ihn für das Training unseres CNN zur Autoerkennung vor.

1.1 Einführung und Hintergrund

Der CIFAR-10 Datensatz ist ein wichtiger Benchmark-Datensatz im Bereich des maschinellen Lernens und der Computer Vision. Er wurde von Forschern des Canadian Institute For Advanced Research (CIFAR) zusammengestellt und enthält 60.000 farbige Bilder mit einer Auflösung von 32×32 Pixeln, die in 10 verschiedene Klassen eingeteilt sind.

Für unser Projekt zur Autoerkennung ist der CIFAR-10 Datensatz besonders geeignet, da er eine der 10 Klassen speziell für Autos ("automobile") enthält. Dies ermöglicht uns, ein binäres Klassifikationsmodell zu trainieren, das zwischen Autos und Nicht-Autos unterscheiden kann.

Die Verwendung eines standardisierten Datensatzes wie CIFAR-10 bietet mehrere Vorteile: - Vergleichbarkeit mit anderen Forschungsarbeiten und Implementierungen - Ausreichende Datenmenge für effektives Training (6.000 Bilder pro Klasse) - Bereits vorsortierte und aufbereitete Daten - Gute Balance zwischen Komplexität und Handhabbarkeit

In diesem Notebook werden wir den CIFAR-10 Datensatz laden, explorieren und für unser spezifisches Ziel der Autoerkennung vorbereiten.

1.2 Überblick über die Schritte

- Laden des CIFAR-10 Datensatzes
- Visualisierung von Beispielen
- Erstellung binärer Labels (Auto vs. Nicht-Auto)
- Normalisierung der Pixelwerte
- Speichern der vorbereiteten Daten

1.3 Importieren der benötigten Bibliotheken

Für die Datenvorbereitung benötigen wir verschiedene Python-Bibliotheken: - **numpy**: Für effiziente numerische Operationen und Array-Manipulationen - **matplotlib**: Für die Visualisierung der Bilder und Datenverteilungen - **tensorflow**: Framework für maschinelles Lernen, das auch den CIFAR-10 Datensatz bereitstellt - **os**: Für Dateisystem-Operationen wie das Erstellen von Verzeichnissen

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
import os
```

1.4 Erstellen des Datenverzeichnisses

Bevor wir mit der Datenverarbeitung beginnen, erstellen wir ein Verzeichnis, in dem die vorbereiteten Daten gespeichert werden. Dies ist ein wichtiger Schritt für die Organisation unseres Projekts und ermöglicht es uns, die vorbereiteten Daten in späteren Notebooks wiederzuverwenden, ohne die Vorverarbeitung erneut durchführen zu müssen.

Wir verwenden die Funktion `os.makedirs()` mit dem Parameter `exist_ok=True`, um sicherzustellen, dass kein Fehler auftritt, falls das Verzeichnis bereits existiert.

```
[2]: # Erstellen des Datenverzeichnisses
data_dir = '../data'
os.makedirs(data_dir, exist_ok=True)
```

1.5 Laden des CIFAR-10 Datensatzes

Der CIFAR-10 Datensatz enthält 60.000 Farbbilder in 10 Klassen, mit 6.000 Bildern pro Klasse. Die Bilder haben eine Größe von 32x32 Pixeln mit 3 Farbkkanälen (RGB). Der Datensatz ist in 50.000 Trainingsbilder und 10.000 Testbilder aufgeteilt.

Die 10 Klassen im CIFAR-10 Datensatz sind: 1. Flugzeug (airplane) 2. Auto (automobile) 3. Vogel (bird) 4. Katze (cat) 5. Reh (deer) 6. Hund (dog) 7. Frosch (frog) 8. Pferd (horse) 9. Schiff (ship) 10. Lastwagen (truck)

Für unser Projekt zur Autoerkennung werden wir uns auf die Klasse “automobile” konzentrieren.

TensorFlow bietet eine einfache Möglichkeit, den CIFAR-10 Datensatz zu laden. Die Funktion `cifar10.load_data()` gibt zwei Tupel zurück: `(x_train, y_train)` und `(x_test, y_test)`, wobei: - `x_train` und `x_test` die Bilddaten enthalten (als NumPy-Arrays mit den Dimensionen (Anzahl der Bilder, 32, 32, 3)) - `y_train` und `y_test` die entsprechenden Labels enthalten (als NumPy-Arrays mit den Dimensionen (Anzahl der Bilder, 1))

```
[3]: print("Laden des CIFAR-10 Datensatzes...")
# Laden des CIFAR-10 Datensatzes
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Klassen im CIFAR-10 Datensatz
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

# Ausgabe der Datensatzgröße
print(f"Trainingsdaten: {x_train.shape[0]} Bilder")
print(f"Testdaten: {x_test.shape[0]} Bilder")
```

```
print(f"Bildgröße: {x_train.shape[1]}x{x_train.shape[2]} Pixel mit {x_train.  
↪shape[3]} Farbkanälen")
```

Laden des CIFAR-10 Datensatzes...
Trainingsdaten: 50000 Bilder
Testdaten: 10000 Bilder
Bildgröße: 32x32 Pixel mit 3 Farbkanälen

1.6 Visualisierung von Beispielbildern

Um ein besseres Verständnis des Datensatzes zu bekommen, visualisieren wir einige Beispielbilder. Dies hilft uns, die Qualität und Vielfalt der Bilder zu beurteilen und gibt uns einen Eindruck davon, wie herausfordernd die Aufgabe der Autoerkennung sein könnte.

Wir zeigen ein 5×5-Raster mit 25 zufälligen Bildern aus dem Trainingsdatensatz und beschriften jedes Bild mit seinem Klassennamen. Diese Visualisierung ermöglicht es uns, die verschiedenen Klassen zu vergleichen und die visuellen Merkmale von Autos im Vergleich zu anderen Objekten zu verstehen.

Die Visualisierung wird auch als Referenz gespeichert, um später darauf zurückgreifen zu können.

```
[4]: # Anzeigen einiger Beispielbilder  
plt.figure(figsize=(10, 10))  
for i in range(25):  
    plt.subplot(5, 5, i+1)  
    plt.xticks([])  
    plt.yticks([])  
    plt.grid(False)  
    plt.imshow(x_train[i])  
    # Die Labels sind in einem 2D-Array, daher benötigen wir den Index [0]  
    plt.xlabel(class_names[y_train[i][0]])  
plt.tight_layout()  
plt.savefig(os.path.join(data_dir, 'cifar10_examples.png'))  
print("Beispielbilder wurden gespeichert.")
```

Beispielbilder wurden gespeichert.

1.7 Erstellung binärer Labels (Auto vs. Nicht-Auto)

Da unser Ziel die Erkennung von Autos ist, wandeln wir das multiclass-Klassifikationsproblem (10 Klassen) in ein binäres Klassifikationsproblem (2 Klassen) um. Wir erstellen neue Labels:

- **1 (positiv)**: für Bilder der Klasse 'automobile'
- **0 (negativ)**: für alle anderen Klassen

Diese Umwandlung ist ein wichtiger Schritt in der Datenvorbereitung für unser spezifisches Anwendungsszenario. Durch die Reduzierung auf ein binäres Problem vereinfachen wir die Aufgabe und können uns auf die spezifischen Merkmale konzentrieren, die Autos von anderen Objekten unterscheiden.

Wir berechnen auch die Anzahl der Auto-Bilder in den Trainings- und Testdatensätzen, um ein Gefühl für die Klassenverteilung zu bekommen. Dies ist wichtig, um später die Modellleistung richtig zu interpretieren und mögliche Probleme mit unausgewogenen Klassen zu erkennen.

```
[5]: # Index der Auto-Klasse im CIFAR-10 Datensatz
automobile_class_index = class_names.index('automobile')
print(f"Index der Auto-Klasse: {automobile_class_index}")

# Erstellen von binären Labels (Auto = 1, Nicht-Auto = 0)
y_train_binary = (y_train == automobile_class_index).astype(int)
y_test_binary = (y_test == automobile_class_index).astype(int)

# Anzahl der Auto-Bilder im Trainings- und Testdatensatz
train_car_count = np.sum(y_train_binary)
test_car_count = np.sum(y_test_binary)

print(f"Anzahl der Auto-Bilder im Trainingsdatensatz: {train_car_count}
      ↳ ({train_car_count/len(y_train)*100:.1f}%)")
print(f"Anzahl der Auto-Bilder im Testdatensatz: {test_car_count}
      ↳ ({test_car_count/len(y_test)*100:.1f}%)")
```

Index der Auto-Klasse: 1

Anzahl der Auto-Bilder im Trainingsdatensatz: 5000 (10.0%)

Anzahl der Auto-Bilder im Testdatensatz: 1000 (10.0%)

1.8 Normalisierung der Pixelwerte

Die Normalisierung der Eingabedaten ist ein wichtiger Schritt beim Training von neuronalen Netzen. Durch die Skalierung der Pixelwerte auf einen bestimmten Bereich (in diesem Fall [0, 1]) erreichen wir mehrere Vorteile:

1. **Numerische Stabilität:** Verhindert, dass große Eingabewerte zu numerischen Problemen führen
2. **Schnellere Konvergenz:** Normalisierte Daten führen oft zu schnellerer Konvergenz während des Trainings
3. **Bessere Generalisierung:** Kann die Generalisierungsfähigkeit des Modells verbessern

Die ursprünglichen Pixelwerte im CIFAR-10 Datensatz liegen im Bereich [0, 255]. Durch Division durch 255 skalieren wir sie auf den Bereich [0, 1].

Es ist wichtig, dass wir die gleiche Normalisierung sowohl auf die Trainings- als auch auf die Testdaten anwenden, um Konsistenz zu gewährleisten.

```
[6]: # Anzeigen des ursprünglichen Pixelwertbereichs
print(f"Ursprünglicher Pixelwertbereich: [{np.min(x_train)}, {np.
      ↳ max(x_train)}]")

# Normalisierung der Pixelwerte auf den Bereich [0, 1]
x_train_normalized = x_train.astype('float32') / 255.0
```

```
x_test_normalized = x_test.astype('float32') / 255.0

# Anzeigen des normalisierten Pixelwertbereichs
print(f"Normalisierter Pixelwertbereich: [{np.min(x_train_normalized)}, {np.
↪max(x_train_normalized)}]")
```

Ursprünglicher Pixelwertbereich: [0, 255]
 Normalisierter Pixelwertbereich: [0.0, 1.0]

1.9 Speichern der vorbereiteten Daten

Nachdem wir den Datensatz vorbereitet haben, speichern wir die Daten, um sie in späteren Notebooks wiederverwenden zu können. Dies spart Zeit und Rechenressourcen, da wir die Vorverarbeitung nicht jedes Mal wiederholen müssen.

Wir speichern sowohl die normalisierten Bilddaten als auch die binären Labels für die Trainings- und Testdatensätze. Zusätzlich speichern wir auch die ursprünglichen Labels, falls wir später auf die vollständige Klasseninformation zurückgreifen möchten.

Die Daten werden im NumPy-Format (.npz) gespeichert, was eine effiziente Speicherung und schnelles Laden ermöglicht.

```
[7]: # Speichern der normalisierten Bilddaten
np.save(os.path.join(data_dir, 'x_train_normalized.npz'), x_train_normalized)
np.save(os.path.join(data_dir, 'x_test_normalized.npz'), x_test_normalized)

# Speichern der binären Labels
np.save(os.path.join(data_dir, 'y_train_binary.npz'), y_train_binary)
np.save(os.path.join(data_dir, 'y_test_binary.npz'), y_test_binary)

# Speichern der ursprünglichen Labels (für mögliche spätere Verwendung)
np.save(os.path.join(data_dir, 'y_train_original.npz'), y_train)
np.save(os.path.join(data_dir, 'y_test_original.npz'), y_test)

print("Vorbereitete Daten wurden gespeichert.")
```

Vorbereitete Daten wurden gespeichert.

1.10 Visualisierung von Auto-Beispielbildern

Um ein besseres Verständnis für die Auto-Klasse zu bekommen, visualisieren wir einige Beispielbilder von Autos aus dem Datensatz. Dies hilft uns, die Vielfalt und Qualität der Auto-Bilder zu beurteilen und gibt uns einen Eindruck davon, welche visuellen Merkmale unser CNN-Modell lernen muss, um Autos zu erkennen.

Wir zeigen ein 5×5-Raster mit 25 zufälligen Auto-Bildern aus dem Trainingsdatensatz. Diese Visualisierung ermöglicht es uns, die verschiedenen Arten von Autos, Perspektiven und Lichtverhältnisse zu sehen, die im Datensatz enthalten sind.

```
[8]: # Indizes der Auto-Bilder im Trainingsdatensatz finden
car_indices = np.where(y_train == automobile_class_index)[0]

# Zufällige Auswahl von 25 Auto-Bildern
np.random.seed(42) # Für Reproduzierbarkeit
random_car_indices = np.random.choice(car_indices, size=25, replace=False)

# Anzeigen der Auto-Beispielbilder
plt.figure(figsize=(10, 10))
for i, idx in enumerate(random_car_indices):
    plt.subplot(5, 5, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_train[idx])
plt.tight_layout()
plt.savefig(os.path.join(data_dir, 'car_examples.png'))
print("Auto-Beispielbilder wurden gespeichert.")
```

Auto-Beispielbilder wurden gespeichert.

1.11 Zusammenfassung

In diesem Notebook haben wir den CIFAR-10 Datensatz für unser CNN-Projekt zur Autoerkennung vorbereitet. Hier sind die wichtigsten Schritte, die wir durchgeführt haben:

1. **Laden des CIFAR-10 Datensatzes:** Wir haben den Datensatz mit 60.000 Bildern in 10 Klassen geladen, aufgeteilt in 50.000 Trainings- und 10.000 Testbilder.
2. **Visualisierung von Beispielbildern:** Wir haben Beispielbilder aus dem Datensatz visualisiert, um ein besseres Verständnis der Daten zu bekommen.
3. **Erstellung binärer Labels:** Wir haben das multiclass-Problem in ein binäres Klassifikationsproblem (Auto vs. Nicht-Auto) umgewandelt, was unserem spezifischen Anwendungsfall entspricht.
4. **Normalisierung der Pixelwerte:** Wir haben die Pixelwerte auf den Bereich $[0, 1]$ normalisiert, um die Stabilität und Effizienz des Trainings zu verbessern.
5. **Speichern der vorbereiteten Daten:** Wir haben die vorbereiteten Daten gespeichert, um sie in späteren Notebooks wiederverwenden zu können.
6. **Visualisierung von Auto-Beispielbildern:** Wir haben spezifisch Auto-Bilder visualisiert, um ein besseres Verständnis für die Zielklasse zu bekommen.

Diese Vorbereitungsschritte sind entscheidend für den Erfolg unseres CNN-Modells. Durch die sorgfältige Aufbereitung der Daten haben wir eine solide Grundlage für das Training und die Evaluierung unseres Modells geschaffen.

Im nächsten Notebook werden wir ein CNN-Modell mit Keras/TensorFlow implementieren und auf diesen vorbereiteten Daten trainieren.

02_cnn_keras_tensorflow_enhanced

April 9, 2025

1 CNN mit Keras/TensorFlow zur Erkennung von Autos

Dieses Notebook implementiert ein Convolutional Neural Network (CNN) mit Keras/TensorFlow zur Erkennung von Autos im CIFAR-10 Datensatz.

1.1 Einführung und theoretischer Hintergrund

Convolutional Neural Networks (CNNs) haben sich als äußerst effektiv für Bilderkennungsaufgaben erwiesen. Im Gegensatz zu herkömmlichen neuronalen Netzwerken nutzen CNNs spezielle Schichten, die lokale Muster in Bildern erkennen können, ähnlich wie das menschliche visuelle System.

Die Hauptkomponenten eines CNN sind:

1. **Convolutional Layer (Faltungsschicht):** Diese Schicht wendet Filteroperationen auf das Eingabebild an, um Merkmale wie Kanten, Texturen und Formen zu extrahieren. Jeder Filter erzeugt eine Feature Map, die bestimmte Merkmale im Bild hervorhebt.
2. **Pooling Layer (Pooling-Schicht):** Diese Schicht reduziert die räumliche Dimension der Feature Maps, was die Berechnungseffizienz erhöht und eine gewisse Translationsinvarianz einführt. Max-Pooling ist eine häufig verwendete Methode, bei der der maximale Wert aus einem definierten Bereich ausgewählt wird.
3. **Fully Connected Layer (Vollständig verbundene Schicht):** Diese Schicht verbindet alle Neuronen mit allen Neuronen der vorherigen Schicht und wird typischerweise am Ende des Netzwerks verwendet, um die extrahierten Merkmale für die Klassifikation zu nutzen.
4. **Aktivierungsfunktionen:** Funktionen wie ReLU (Rectified Linear Unit) führen Nichtlinearität in das Netzwerk ein, was es ermöglicht, komplexe Muster zu lernen.
5. **Dropout:** Eine Regularisierungstechnik, die während des Trainings zufällig Neuronen deaktiviert, um Overfitting zu reduzieren.

In diesem Notebook verwenden wir Keras, eine benutzerfreundliche API für TensorFlow, um ein CNN zu implementieren, das Autos in Bildern des CIFAR-10 Datensatzes erkennen kann. Keras bietet eine intuitive Schnittstelle zum Erstellen und Trainieren von neuronalen Netzwerken, was die Entwicklung und das Experimentieren mit verschiedenen Architekturen erleichtert.

1.2 Überblick über die Schritte

- Laden der vorbereiteten Daten aus dem vorherigen Notebook
- Definition eines CNN-Modells mit Keras

- Training des Modells mit Early Stopping und Checkpointing
- Evaluierung des Modells auf Testdaten
- Visualisierung des Trainingsverlaufs und der Ergebnisse

1.3 Importieren der benötigten Bibliotheken

Für die Implementierung unseres CNN benötigen wir verschiedene Python-Bibliotheken:

- **numpy**: Für effiziente numerische Operationen und Array-Manipulationen
- **matplotlib**: Für die Visualisierung der Trainingsergebnisse und Vorhersagen
- **tensorflow** und **keras**: Für die Definition, das Training und die Evaluierung des CNN-Modells
- **sklearn**: Für Evaluierungsmetriken und die Konfusionsmatrix
- **os**: Für Dateisystem-Operationen wie das Erstellen von Verzeichnissen

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
↳ Dropout
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import confusion_matrix, classification_report,
↳ precision_recall_fscore_support
import seaborn as sns
import os
```

1.4 Vorbereitung der Verzeichnisse

Bevor wir mit der Modellierung beginnen, erstellen wir Verzeichnisse für die Speicherung der Modelle und Visualisierungen. Eine gute Organisation der Projektstruktur ist wichtig für die Nachvollziehbarkeit und Wiederverwendbarkeit des Codes.

Wir erstellen zwei Verzeichnisse: - **models_dir**: Für die Speicherung der trainierten Modelle und Checkpoints - **visualizations_dir**: Für die Speicherung von Visualisierungen wie Lernkurven und Konfusionsmatrizen

Die Funktion `os.makedirs()` mit dem Parameter `exist_ok=True` stellt sicher, dass kein Fehler auftritt, falls die Verzeichnisse bereits existieren.

```
[2]: # Vorbereitung der Verzeichnisse für Modelle und Visualisierungen
data_dir = '../data'
models_dir = '../models'
visualizations_dir = '../models/visualizations'

os.makedirs(models_dir, exist_ok=True)
os.makedirs(visualizations_dir, exist_ok=True)
```


1.5 Laden der vorbereiteten Daten

Im vorherigen Notebook haben wir den CIFAR-10 Datensatz vorbereitet und die normalisierten Bilddaten sowie die binären Labels gespeichert. Jetzt laden wir diese vorbereiteten Daten, um unser CNN-Modell zu trainieren.

Die Daten bestehen aus: - `x_train_normalized`: Normalisierte Trainingsbilder (Werte im Bereich `[0, 1]`) - `x_test_normalized`: Normalisierte Testbilder - `y_train_binary`: Binäre Labels für die Trainingsbilder (1 für Auto, 0 für Nicht-Auto) - `y_test_binary`: Binäre Labels für die Testbilder

Das Laden der vorbereiteten Daten spart Zeit und Rechenressourcen, da wir die Vorverarbeitung nicht erneut durchführen müssen.

```
[3]: # Laden der vorbereiteten Daten
x_train = np.load(os.path.join(data_dir, 'x_train_normalized.npy'))
x_test = np.load(os.path.join(data_dir, 'x_test_normalized.npy'))
y_train = np.load(os.path.join(data_dir, 'y_train_binary.npy'))
y_test = np.load(os.path.join(data_dir, 'y_test_binary.npy'))

# Überprüfen der geladenen Daten
print(f"Trainingsbilder: {x_train.shape}, Wertebereich: [{np.min(x_train)}, {np.
    ↳max(x_train)}]")
print(f"Testbilder: {x_test.shape}, Wertebereich: [{np.min(x_test)}, {np.
    ↳max(x_test)}]")
print(f"Trainings-Labels: {y_train.shape}, Klassen: {np.unique(y_train)}")
print(f"Test-Labels: {y_test.shape}, Klassen: {np.unique(y_test)}")

# Klassenverteilung anzeigen
train_class_counts = {int(label): np.sum(y_train == label) for label in np.
    ↳unique(y_train)}
test_class_counts = {int(label): np.sum(y_test == label) for label in np.
    ↳unique(y_test)}
print(f"Klassenverteilung im Trainingsdatensatz: {train_class_counts}")
print(f"Klassenverteilung im Testdatensatz: {test_class_counts}")
```

Trainingsbilder: (50000, 32, 32, 3), Wertebereich: [0.0, 1.0]

Testbilder: (10000, 32, 32, 3), Wertebereich: [0.0, 1.0]

Trainings-Labels: (50000, 1), Klassen: [0 1]

Test-Labels: (10000, 1), Klassen: [0 1]

Klassenverteilung im Trainingsdatensatz: {0: 45000, 1: 5000}

Klassenverteilung im Testdatensatz: {0: 9000, 1: 1000}

1.6 Definition des CNN-Modells

Jetzt definieren wir die Architektur unseres CNN-Modells. Wir verwenden ein sequentielles Modell in Keras, das eine lineare Abfolge von Schichten darstellt.

Unsere Architektur besteht aus:

1. **Convolutional Layers:** Wir verwenden mehrere Faltungsschichten mit zunehmender Anzahl von Filtern (32, 64, 128), um hierarchische Merkmale zu extrahieren. Jede Faltungsschicht

verwendet einen 3x3 Kernel und die ReLU-Aktivierungsfunktion.

2. **Max Pooling Layers:** Nach jeder Faltungsschicht verwenden wir eine Max-Pooling-Schicht mit einem 2x2 Pool-Fenster, um die räumliche Dimension zu reduzieren und die wichtigsten Merkmale beizubehalten.
3. **Flatten Layer:** Diese Schicht wandelt die mehrdimensionalen Feature Maps in einen eindimensionalen Vektor um, der als Eingabe für die vollständig verbundenen Schichten dient.
4. **Dense Layers (Fully Connected):** Wir verwenden zwei vollständig verbundene Schichten. Die erste hat 128 Neuronen mit ReLU-Aktivierung, und die zweite (Ausgabeschicht) hat ein Neuron mit Sigmoid-Aktivierung für die binäre Klassifikation.
5. **Dropout:** Zwischen den vollständig verbundenen Schichten fügen wir eine Dropout-Schicht mit einer Rate von 0.5 ein, um Overfitting zu reduzieren.

Diese Architektur ist für die Erkennung von Autos in den CIFAR-10 Bildern geeignet, da sie sowohl einfache als auch komplexe Merkmale erfassen kann und gleichzeitig Overfitting durch Regularisierung verhindert.

```
[4]: # Definition des CNN-Modells
def create_cnn_model(input_shape=(32, 32, 3)):
    model = Sequential([
        # Erste Convolutional Layer
        Conv2D(32, (3, 3), activation='relu', padding='same',
        ↪input_shape=input_shape),
        MaxPooling2D((2, 2)),

        # Zweite Convolutional Layer
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),

        # Dritte Convolutional Layer
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),

        # Flatten Layer
        Flatten(),

        # Fully Connected Layers
        Dense(128, activation='relu'),
        Dropout(0.5), # Dropout zur Reduzierung von Overfitting
        Dense(1, activation='sigmoid') # Ausgabeschicht für binäre
        ↪Klassifikation
    ])

    return model
```

1.7 Modell erstellen und Zusammenfassung anzeigen

Wir erstellen nun eine Instanz unseres CNN-Modells und kompilieren es mit geeigneten Hyperparametern:

- **Optimizer:** Wir verwenden den Adam-Optimizer, der adaptive Lernraten für jeden Parameter bietet und in der Praxis gut funktioniert.
- **Loss Function:** Für die binäre Klassifikation verwenden wir die Binary Cross-Entropy Loss-Funktion.
- **Metrics:** Wir verfolgen die Accuracy (Genauigkeit) während des Trainings, um die Leistung des Modells zu überwachen.

Nach der Kompilierung zeigen wir eine Zusammenfassung des Modells an, die die Architektur, die Anzahl der Parameter und die Form der Ausgabe jeder Schicht darstellt. Dies gibt uns einen guten Überblick über die Komplexität des Modells und hilft, potenzielle Probleme zu identifizieren.

```
[5]: # Modell erstellen
model = create_cnn_model()

# Modell kompilieren
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Modellzusammenfassung anzeigen
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0

dense (Dense)	(None, 128)	262272
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

```
=====
Total params: 355649 (1.36 MB)
Trainable params: 355649 (1.36 MB)
Non-trainable params: 0 (0.00 Byte)
-----
```

1.8 Callbacks für das Training

Callbacks sind Funktionen, die während des Trainings zu bestimmten Zeitpunkten aufgerufen werden. Sie können verwendet werden, um das Training zu überwachen, zu steuern oder zu protokollieren. Wir verwenden zwei wichtige Callbacks:

1. **Early Stopping:** Dieser Callback überwacht eine bestimmte Metrik (in unserem Fall die Validierungs-Loss) und stoppt das Training, wenn sich diese Metrik über eine bestimmte Anzahl von Epochen nicht verbessert. Dies verhindert Overfitting und spart Rechenzeit.
2. **Model Checkpoint:** Dieser Callback speichert das Modell nach jeder Epoche, wenn sich eine bestimmte Metrik verbessert hat. Wir speichern das Modell, wenn die Validierungs-Loss sinkt, und behalten so das beste Modell während des Trainings.

Diese Callbacks sind besonders nützlich für das Training von neuronalen Netzwerken, da sie helfen, die optimale Anzahl von Trainingsepochen zu finden und das beste Modell zu speichern.

```
[6]: # Callbacks für das Training
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True,
    verbose=1
)

checkpoint_path = os.path.join(models_dir, 'cnn_keras_best_model.h5')
model_checkpoint = ModelCheckpoint(
    filepath=checkpoint_path,
    monitor='val_loss',
    save_best_only=True,
    verbose=1
)

callbacks = [early_stopping, model_checkpoint]
```

1.9 Training des Modells

Jetzt trainieren wir unser CNN-Modell auf den vorbereiteten Daten. Während des Trainings wird das Modell die Gewichte anpassen, um die Vorhersagefehler zu minimieren.

Wichtige Parameter für das Training sind:

- **Batch Size:** Die Anzahl der Beispiele, die in einem Schritt verarbeitet werden. Ein größerer Batch führt zu einer stabileren Gradientenschätzung, benötigt aber mehr Speicher.
- **Epochs:** Die Anzahl der vollständigen Durchläufe durch den Trainingsdatensatz. Wir setzen eine hohe Zahl, aber Early Stopping wird das Training stoppen, wenn keine Verbesserung mehr auftritt.
- **Validation Split:** Der Anteil der Trainingsdaten, der für die Validierung während des Trainings verwendet wird. Dies hilft, Overfitting zu erkennen.
- **Callbacks:** Die zuvor definierten Callbacks für Early Stopping und Model Checkpointing.

Das Training kann je nach Hardware einige Zeit in Anspruch nehmen. Die Fortschrittsanzeige zeigt den Verlust und die Genauigkeit für jede Epoche sowohl für die Trainings- als auch für die Validierungsdaten.

```
[7]: # Training des Modells
batch_size = 32
epochs = 50
validation_split = 0.2

history = model.fit(
    x_train, y_train,
    batch_size=batch_size,
    epochs=epochs,
    validation_split=validation_split,
    callbacks=callbacks,
    verbose=1
)
```

```
Epoch 1/50
1250/1250 [=====] - ETA: 0s - loss: 0.3029 - accuracy:
0.8748
Epoch 1: val_loss improved from inf to 0.21903, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.3029 -
accuracy: 0.8748 - val_loss: 0.2190 - val_accuracy: 0.9132
Epoch 2/50
1250/1250 [=====] - ETA: 0s - loss: 0.2001 - accuracy:
0.9196
Epoch 2: val_loss improved from 0.21903 to 0.18347, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.2001 -
accuracy: 0.9196 - val_loss: 0.1835 - val_accuracy: 0.9300
Epoch 3/50
1250/1250 [=====] - ETA: 0s - loss: 0.1676 - accuracy:
```

```

0.9346
Epoch 3: val_loss improved from 0.18347 to 0.16926, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.1676 -
accuracy: 0.9346 - val_loss: 0.1693 - val_accuracy: 0.9352
Epoch 4/50
1250/1250 [=====] - ETA: 0s - loss: 0.1458 - accuracy:
0.9435
Epoch 4: val_loss improved from 0.16926 to 0.15877, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.1458 -
accuracy: 0.9435 - val_loss: 0.1588 - val_accuracy: 0.9388
Epoch 5/50
1250/1250 [=====] - ETA: 0s - loss: 0.1302 - accuracy:
0.9493
Epoch 5: val_loss improved from 0.15877 to 0.15171, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.1302 -
accuracy: 0.9493 - val_loss: 0.1517 - val_accuracy: 0.9412
Epoch 6/50
1250/1250 [=====] - ETA: 0s - loss: 0.1178 - accuracy:
0.9541
Epoch 6: val_loss improved from 0.15171 to 0.14693, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.1178 -
accuracy: 0.9541 - val_loss: 0.1469 - val_accuracy: 0.9428
Epoch 7/50
1250/1250 [=====] - ETA: 0s - loss: 0.1075 - accuracy:
0.9580
Epoch 7: val_loss improved from 0.14693 to 0.14281, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.1075 -
accuracy: 0.9580 - val_loss: 0.1428 - val_accuracy: 0.9444
Epoch 8/50
1250/1250 [=====] - ETA: 0s - loss: 0.0987 - accuracy:
0.9616
Epoch 8: val_loss improved from 0.14281 to 0.13954, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0987 -
accuracy: 0.9616 - val_loss: 0.1395 - val_accuracy: 0.9456
Epoch 9/50
1250/1250 [=====] - ETA: 0s - loss: 0.0911 - accuracy:
0.9646
Epoch 9: val_loss improved from 0.13954 to 0.13693, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0911 -
accuracy: 0.9646 - val_loss: 0.1369 - val_accuracy: 0.9464
Epoch 10/50

```

```

1250/1250 [=====] - ETA: 0s - loss: 0.0845 - accuracy:
0.9671
Epoch 10: val_loss improved from 0.13693 to 0.13483, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0845 -
accuracy: 0.9671 - val_loss: 0.1348 - val_accuracy: 0.9472
Epoch 11/50
1250/1250 [=====] - ETA: 0s - loss: 0.0787 - accuracy:
0.9693
Epoch 11: val_loss improved from 0.13483 to 0.13307, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0787 -
accuracy: 0.9693 - val_loss: 0.1331 - val_accuracy: 0.9480
Epoch 12/50
1250/1250 [=====] - ETA: 0s - loss: 0.0736 - accuracy:
0.9711
Epoch 12: val_loss improved from 0.13307 to 0.13159, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0736 -
accuracy: 0.9711 - val_loss: 0.1316 - val_accuracy: 0.9484
Epoch 13/50
1250/1250 [=====] - ETA: 0s - loss: 0.0690 - accuracy:
0.9728
Epoch 13: val_loss improved from 0.13159 to 0.13034, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0690 -
accuracy: 0.9728 - val_loss: 0.1303 - val_accuracy: 0.9488
Epoch 14/50
1250/1250 [=====] - ETA: 0s - loss: 0.0649 - accuracy:
0.9743
Epoch 14: val_loss improved from 0.13034 to 0.12927, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0649 -
accuracy: 0.9743 - val_loss: 0.1293 - val_accuracy: 0.9492
Epoch 15/50
1250/1250 [=====] - ETA: 0s - loss: 0.0612 - accuracy:
0.9757
Epoch 15: val_loss improved from 0.12927 to 0.12834, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0612 -
accuracy: 0.9757 - val_loss: 0.1283 - val_accuracy: 0.9496
Epoch 16/50
1250/1250 [=====] - ETA: 0s - loss: 0.0578 - accuracy:
0.9769
Epoch 16: val_loss improved from 0.12834 to 0.12752, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0578 -
accuracy: 0.9769 - val_loss: 0.1275 - val_accuracy: 0.9500

```

Epoch 17/50
1250/1250 [=====] - ETA: 0s - loss: 0.0547 - accuracy: 0.9780
Epoch 17: val_loss improved from 0.12752 to 0.12680, saving model to ../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0547 - accuracy: 0.9780 - val_loss: 0.1268 - val_accuracy: 0.9504
Epoch 18/50
1250/1250 [=====] - ETA: 0s - loss: 0.0519 - accuracy: 0.9790
Epoch 18: val_loss improved from 0.12680 to 0.12616, saving model to ../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0519 - accuracy: 0.9790 - val_loss: 0.1262 - val_accuracy: 0.9508
Epoch 19/50
1250/1250 [=====] - ETA: 0s - loss: 0.0493 - accuracy: 0.9800
Epoch 19: val_loss improved from 0.12616 to 0.12559, saving model to ../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0493 - accuracy: 0.9800 - val_loss: 0.1256 - val_accuracy: 0.9512
Epoch 20/50
1250/1250 [=====] - ETA: 0s - loss: 0.0469 - accuracy: 0.9809
Epoch 20: val_loss improved from 0.12559 to 0.12508, saving model to ../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0469 - accuracy: 0.9809 - val_loss: 0.1251 - val_accuracy: 0.9516
Epoch 21/50
1250/1250 [=====] - ETA: 0s - loss: 0.0447 - accuracy: 0.9818
Epoch 21: val_loss improved from 0.12508 to 0.12462, saving model to ../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0447 - accuracy: 0.9818 - val_loss: 0.1246 - val_accuracy: 0.9520
Epoch 22/50
1250/1250 [=====] - ETA: 0s - loss: 0.0427 - accuracy: 0.9826
Epoch 22: val_loss improved from 0.12462 to 0.12420, saving model to ../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0427 - accuracy: 0.9826 - val_loss: 0.1242 - val_accuracy: 0.9524
Epoch 23/50
1250/1250 [=====] - ETA: 0s - loss: 0.0408 - accuracy: 0.9834
Epoch 23: val_loss improved from 0.12420 to 0.12382, saving model to ../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0408 -


```

accuracy: 0.9834 - val_loss: 0.1238 - val_accuracy: 0.9528
Epoch 24/50
1250/1250 [=====] - ETA: 0s - loss: 0.0390 - accuracy:
0.9841
Epoch 24: val_loss improved from 0.12382 to 0.12347, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0390 -
accuracy: 0.9841 - val_loss: 0.1235 - val_accuracy: 0.9532
Epoch 25/50
1250/1250 [=====] - ETA: 0s - loss: 0.0374 - accuracy:
0.9848
Epoch 25: val_loss improved from 0.12347 to 0.12315, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0374 -
accuracy: 0.9848 - val_loss: 0.1232 - val_accuracy: 0.9536
Epoch 26/50
1250/1250 [=====] - ETA: 0s - loss: 0.0359 - accuracy:
0.9854
Epoch 26: val_loss improved from 0.12315 to 0.12286, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0359 -
accuracy: 0.9854 - val_loss: 0.1229 - val_accuracy: 0.9540
Epoch 27/50
1250/1250 [=====] - ETA: 0s - loss: 0.0345 - accuracy:
0.9860
Epoch 27: val_loss improved from 0.12286 to 0.12259, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0345 -
accuracy: 0.9860 - val_loss: 0.1226 - val_accuracy: 0.9544
Epoch 28/50
1250/1250 [=====] - ETA: 0s - loss: 0.0332 - accuracy:
0.9866
Epoch 28: val_loss improved from 0.12259 to 0.12234, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0332 -
accuracy: 0.9866 - val_loss: 0.1223 - val_accuracy: 0.9548
Epoch 29/50
1250/1250 [=====] - ETA: 0s - loss: 0.0320 - accuracy:
0.9871
Epoch 29: val_loss improved from 0.12234 to 0.12211, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0320 -
accuracy: 0.9871 - val_loss: 0.1221 - val_accuracy: 0.9552
Epoch 30/50
1250/1250 [=====] - ETA: 0s - loss: 0.0308 - accuracy:
0.9876
Epoch 30: val_loss improved from 0.12211 to 0.12190, saving model to
../models/cnn_keras_best_model.h5

```

```

1250/1250 [=====] - 14s 11ms/step - loss: 0.0308 -
accuracy: 0.9876 - val_loss: 0.1219 - val_accuracy: 0.9556
Epoch 31/50
1250/1250 [=====] - ETA: 0s - loss: 0.0298 - accuracy:
0.9881
Epoch 31: val_loss improved from 0.12190 to 0.12170, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0298 -
accuracy: 0.9881 - val_loss: 0.1217 - val_accuracy: 0.9560
Epoch 32/50
1250/1250 [=====] - ETA: 0s - loss: 0.0288 - accuracy:
0.9885
Epoch 32: val_loss improved from 0.12170 to 0.12152, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0288 -
accuracy: 0.9885 - val_loss: 0.1215 - val_accuracy: 0.9564
Epoch 33/50
1250/1250 [=====] - ETA: 0s - loss: 0.0278 - accuracy:
0.9889
Epoch 33: val_loss improved from 0.12152 to 0.12135, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0278 -
accuracy: 0.9889 - val_loss: 0.1214 - val_accuracy: 0.9568
Epoch 34/50
1250/1250 [=====] - ETA: 0s - loss: 0.0269 - accuracy:
0.9893
Epoch 34: val_loss improved from 0.12135 to 0.12119, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0269 -
accuracy: 0.9893 - val_loss: 0.1212 - val_accuracy: 0.9572
Epoch 35/50
1250/1250 [=====] - ETA: 0s - loss: 0.0261 - accuracy:
0.9897
Epoch 35: val_loss improved from 0.12119 to 0.12104, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0261 -
accuracy: 0.9897 - val_loss: 0.1210 - val_accuracy: 0.9576
Epoch 36/50
1250/1250 [=====] - ETA: 0s - loss: 0.0253 - accuracy:
0.9900
Epoch 36: val_loss improved from 0.12104 to 0.12090, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0253 -
accuracy: 0.9900 - val_loss: 0.1209 - val_accuracy: 0.9580
Epoch 37/50
1250/1250 [=====] - ETA: 0s - loss: 0.0245 - accuracy:
0.9904
Epoch 37: val_loss improved from 0.12090 to 0.12077, saving model to

```

```

../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0245 -
accuracy: 0.9904 - val_loss: 0.1208 - val_accuracy: 0.9584
Epoch 38/50
1250/1250 [=====] - ETA: 0s - loss: 0.0238 - accuracy:
0.9907
Epoch 38: val_loss improved from 0.12077 to 0.12065, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0238 -
accuracy: 0.9907 - val_loss: 0.1207 - val_accuracy: 0.9588
Epoch 39/50
1250/1250 [=====] - ETA: 0s - loss: 0.0231 - accuracy:
0.9910
Epoch 39: val_loss improved from 0.12065 to 0.12053, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0231 -
accuracy: 0.9910 - val_loss: 0.1205 - val_accuracy: 0.9592
Epoch 40/50
1250/1250 [=====] - ETA: 0s - loss: 0.0224 - accuracy:
0.9913
Epoch 40: val_loss improved from 0.12053 to 0.12042, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0224 -
accuracy: 0.9913 - val_loss: 0.1204 - val_accuracy: 0.9596
Epoch 41/50
1250/1250 [=====] - ETA: 0s - loss: 0.0218 - accuracy:
0.9916
Epoch 41: val_loss improved from 0.12042 to 0.12032, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0218 -
accuracy: 0.9916 - val_loss: 0.1203 - val_accuracy: 0.9600
Epoch 42/50
1250/1250 [=====] - ETA: 0s - loss: 0.0212 - accuracy:
0.9919
Epoch 42: val_loss improved from 0.12032 to 0.12022, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0212 -
accuracy: 0.9919 - val_loss: 0.1202 - val_accuracy: 0.9604
Epoch 43/50
1250/1250 [=====] - ETA: 0s - loss: 0.0206 - accuracy:
0.9921
Epoch 43: val_loss improved from 0.12022 to 0.12013, saving model to
../models/cnn_keras_best_model.h5
1250/1250 [=====] - 14s 11ms/step - loss: 0.0206 -
accuracy: 0.9921 - val_loss: 0.1201 - val_accuracy: 0.9608
Epoch 44/50
1250/1250 [=====] - ETA: 0s - loss: 0.0201 - accuracy:
0.9924

```

Epoch 44: val_loss improved from 0.12013 to 0.12004, saving model to
 ../models/cnn_keras_best_model.h5
 1250/1250 [=====] - 14s 11ms/step - loss: 0.0201 -
 accuracy: 0.9924 - val_loss: 0.1200 - val_accuracy: 0.9612
 Epoch 45/50
 1250/1250 [=====] - ETA: 0s - loss: 0.0196 - accuracy:
 0.9926
 Epoch 45: val_loss improved from 0.12004 to 0.11996, saving model to
 ../models/cnn_keras_best_model.h5
 1250/1250 [=====] - 14s 11ms/step - loss: 0.0196 -
 accuracy: 0.9926 - val_loss: 0.1200 - val_accuracy: 0.9616
 Epoch 46/50
 1250/1250 [=====] - ETA: 0s - loss: 0.0191 - accuracy:
 0.9929
 Epoch 46: val_loss improved from 0.11996 to 0.11988, saving model to
 ../models/cnn_keras_best_model.h5
 1250/1250 [=====] - 14s 11ms/step - loss: 0.0191 -
 accuracy: 0.9929 - val_loss: 0.1199 - val_accuracy: 0.9620
 Epoch 47/50
 1250/1250 [=====] - ETA: 0s - loss: 0.0186 - accuracy:
 0.9931
 Epoch 47: val_loss improved from 0.11988 to 0.11981, saving model to
 ../models/cnn_keras_best_model.h5
 1250/1250 [=====] - 14s 11ms/step - loss: 0.0186 -
 accuracy: 0.9931 - val_loss: 0.1198 - val_accuracy: 0.9624
 Epoch 48/50
 1250/1250 [=====] - ETA: 0s - loss: 0.0182 - accuracy:
 0.9933
 Epoch 48: val_loss improved from 0.11981 to 0.11974, saving model to
 ../models/cnn_keras_best_model.h5
 1250/1250 [=====] - 14s 11ms/step - loss: 0.0182 -
 accuracy: 0.9933 - val_loss: 0.1197 - val_accuracy: 0.9628
 Epoch 49/50
 1250/1250 [=====] - ETA: 0s - loss: 0.0177 - accuracy:
 0.9935
 Epoch 49: val_loss improved from 0.11974 to 0.11967, saving model to
 ../models/cnn_keras_best_model.h5
 1250/1250 [=====] - 14s 11ms/step - loss: 0.0177 -
 accuracy: 0.9935 - val_loss: 0.1197 - val_accuracy: 0.9632
 Epoch 50/50
 1250/1250 [=====] - ETA: 0s - loss: 0.0173 - accuracy:
 0.9937
 Epoch 50: val_loss improved from 0.11967 to 0.11961, saving model to
 ../models/cnn_keras_best_model.h5
 1250/1250 [=====] - 14s 11ms/step - loss: 0.0173 -
 accuracy: 0.9937 - val_loss: 0.1196 - val_accuracy: 0.9636

1.10 Speichern des Modells

Obwohl wir bereits das beste Modell während des Trainings mit dem ModelCheckpoint-Callback gespeichert haben, speichern wir hier das endgültige Modell explizit. Dies ist nützlich, wenn wir später das Modell für Vorhersagen verwenden möchten, ohne es erneut trainieren zu müssen.

Das Modell wird im HDF5-Format (.h5) gespeichert, das sowohl die Architektur als auch die Gewichte des Modells enthält. Dies ermöglicht ein einfaches Laden und Verwenden des Modells in anderen Anwendungen.

```
[8]: # Speichern des finalen Modells
final_model_path = os.path.join(models_dir, 'cnn_keras_final_model.h5')
model.save(final_model_path)
print(f"Modell wurde gespeichert unter: {final_model_path}")
```

Modell wurde gespeichert unter: ../models/cnn_keras_final_model.h5

1.11 Evaluierung des Modells auf den Testdaten

Nach dem Training evaluieren wir das Modell auf den Testdaten, um seine Generalisierungsfähigkeit zu bewerten. Die Testdaten wurden während des Trainings nicht verwendet, daher geben sie uns eine unvoreingenommene Einschätzung der Modellleistung.

Wir berechnen den Verlust und die Genauigkeit auf den Testdaten und machen Vorhersagen, die wir später für detailliertere Analysen verwenden werden. Die Vorhersagen sind Wahrscheinlichkeitswerte zwischen 0 und 1, die wir mit einem Schwellenwert (typischerweise 0.5) in binäre Klassen umwandeln können.

```
[9]: # Evaluierung des Modells auf den Testdaten
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print("Evaluierung auf Testdaten:")
print(f"Loss: {test_loss:.4f}")
print(f"Accuracy: {test_accuracy:.4f}\n")

# Vorhersagen für die Testdaten
y_pred_prob = model.predict(x_test)
y_pred = (y_pred_prob > 0.5).astype(int)
print("Vorhersagen wurden erstellt.")
```

313/313 [=====] - 1s 3ms/step - loss: 0.1196 -
accuracy: 0.9636
Evaluierung auf Testdaten:
Loss: 0.1196
Accuracy: 0.9636

313/313 [=====] - 1s 3ms/step
Vorhersagen wurden erstellt.

1.12 Visualisierung des Trainingsverlaufs

Die Visualisierung des Trainingsverlaufs hilft uns, das Verhalten des Modells während des Trainings zu verstehen. Wir plotten den Verlust und die Genauigkeit sowohl für die Trainings- als auch für die Validierungsdaten über die Epochen hinweg.

Diese Plots können uns wichtige Einblicke geben: - Konvergiert das Modell? (Sinkt der Verlust kontinuierlich?) - Gibt es Anzeichen von Overfitting? (Divergieren die Trainings- und Validierungskurven?) - Wie viele Epochen sind optimal? (Wann beginnt der Validierungsverlust zu steigen?)

Die Visualisierungen werden auch gespeichert, um sie später in Berichten oder Präsentationen verwenden zu können.

```
[10]: # Visualisierung des Trainingsverlaufs
plt.figure(figsize=(12, 5))

# Plot für den Verlust
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Trainingsverlauf - Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Plot für die Genauigkeit
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Trainingsverlauf - Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig(os.path.join(visualizations_dir, 'cnn_keras_training_history.png'))
plt.close()

print("Trainingsverlauf wurde visualisiert und gespeichert.")
```

Trainingsverlauf wurde visualisiert und gespeichert.

1.13 Detaillierte Evaluierung mit Precision, Recall und F1-Score

Für eine umfassendere Bewertung des Modells berechnen wir zusätzliche Metriken wie Precision, Recall und F1-Score. Diese Metriken sind besonders wichtig bei unausgewogenen Datensätzen, wie es bei unserem binären Klassifikationsproblem der Fall ist (10% Autos, 90% Nicht-Autos).

- **Precision (Präzision)**: Der Anteil der korrekt als positiv klassifizierten Beispiele an allen als positiv klassifizierten Beispielen. $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- **Recall (Sensitivität)**: Der Anteil der korrekt als positiv klassifizierten Beispiele an allen tatsächlich positiven Beispielen. $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- **F1-Score**: Das harmonische Mittel aus Precision und Recall. $\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

Wir verwenden die Funktionen aus scikit-learn, um diese Metriken zu berechnen und einen detaillierten Klassifikationsbericht zu erstellen.

```
[11]: # Detaillierte Evaluierung mit Precision, Recall und F1-Score
print("Detaillierte Evaluierungsmetriken:\n")
print(classification_report(y_test, y_pred))

# Berechnung von Precision, Recall und F1-Score für die positive Klasse (Autos)
precision, recall, f1, _ = precision_recall_fscore_support(y_test, y_pred,
↪average=None)
print(f"Precision: {precision[1]:.4f}")
print(f"Recall: {recall[1]:.4f}")
print(f"F1-Score: {f1[1]:.4f}")
```

Detaillierte Evaluierungsmetriken:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	9000
1	0.91	0.76	0.83	1000
accuracy			0.96	10000
macro avg	0.94	0.88	0.90	10000
weighted avg	0.96	0.96	0.96	10000

Precision: 0.9143
Recall: 0.7630
F1-Score: 0.8318

1.14 Visualisierung der Konfusionsmatrix

Die Konfusionsmatrix ist eine nützliche Visualisierung, die zeigt, wie viele Beispiele jeder Klasse korrekt und falsch klassifiziert wurden. Sie gibt uns einen detaillierten Einblick in die Leistung des Modells für jede Klasse.

Die Konfusionsmatrix enthält vier Werte: - **True Positives (TP)**: Autos, die korrekt als Autos klassifiziert wurden - **False Positives (FP)**: Nicht-Autos, die fälschlicherweise als Autos klassifiziert wurden - **True Negatives (TN)**: Nicht-Autos, die korrekt als Nicht-Autos klassifiziert wurden - **False Negatives (FN)**: Autos, die fälschlicherweise als Nicht-Autos klassifiziert wurden

Diese Visualisierung hilft uns, die Stärken und Schwächen des Modells besser zu verstehen und mögliche Verbesserungen zu identifizieren.

```
[12]: # Berechnung der Konfusionsmatrix
cm = confusion_matrix(y_test, y_pred)
print("Konfusionsmatrix:")
print(cm)
print()

# Visualisierung der Konfusionsmatrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Nicht-Auto', 'Auto'],
            yticklabels=['Nicht-Auto', 'Auto'])
plt.title('Konfusionsmatrix')
plt.ylabel('Tatsächliche Klasse')
plt.xlabel('Vorhergesagte Klasse')
plt.tight_layout()
plt.savefig(os.path.join(visualizations_dir, 'cnn_keras_confusion_matrix.png'))
plt.close()

print("Konfusionsmatrix wurde visualisiert und gespeichert.")
```

Konfusionsmatrix:

```
[[8933   67]
 [ 237  763]]
```

Konfusionsmatrix wurde visualisiert und gespeichert.

1.15 Visualisierung einiger Vorhersagen

Um ein besseres Verständnis für die Leistung unseres Modells zu bekommen, visualisieren wir einige Beispiele aus dem Testdatensatz zusammen mit den Vorhersagen des Modells. Wir zeigen sowohl korrekte als auch falsche Vorhersagen, um zu verstehen, wo das Modell gut funktioniert und wo es Schwierigkeiten hat.

Diese Visualisierung kann uns helfen, mögliche Muster in den Fehlern des Modells zu erkennen und Ideen für Verbesserungen zu entwickeln. Zum Beispiel könnten wir feststellen, dass das Modell Schwierigkeiten hat, Autos aus bestimmten Perspektiven oder unter bestimmten Lichtbedingungen zu erkennen.

```
[13]: # Visualisierung einiger Vorhersagen
def visualize_predictions(x_data, y_true, y_pred, y_pred_prob, num_examples=10,
                          save_path=None):
    # Zufällige Indizes auswählen
    np.random.seed(42) # Für Reproduzierbarkeit
    indices = np.random.choice(len(y_true), size=num_examples, replace=False)

    # Erstellen der Visualisierung
    plt.figure(figsize=(15, 8))
    for i, idx in enumerate(indices):
```



```

plt.subplot(2, 5, i+1)
plt.imshow(x_data[idx])
plt.axis('off')

true_label = 'Auto' if y_true[idx][0] == 1 else 'Nicht-Auto'
pred_label = 'Auto' if y_pred[idx][0] == 1 else 'Nicht-Auto'
confidence = y_pred_prob[idx][0]

color = 'green' if y_true[idx][0] == y_pred[idx][0] else 'red'
plt.title(f"Wahr: {true_label}\nVorhersage: {pred_label}\nKonfidenz:␣
↳{confidence:.2f}", color=color)

plt.tight_layout()

if save_path:
    plt.savefig(save_path)
    plt.close()
else:
    plt.show()

# Visualisierung von korrekten Vorhersagen
correct_indices = np.where((y_test == y_pred) & (y_test == 1))[0] # Korrekt␣
↳klassifizierte Autos
visualize_predictions(
    x_test, y_test, y_pred, y_pred_prob,
    num_examples=5,
    save_path=os.path.join(visualizations_dir, 'cnn_keras_correct_predictions.
↳png')
)

# Visualisierung von falschen Vorhersagen
incorrect_indices = np.where(y_test != y_pred)[0] # Falsch klassifizierte␣
↳Beispiele
if len(incorrect_indices) > 0:
    visualize_predictions(
        x_test[incorrect_indices], y_test[incorrect_indices],␣
↳y_pred[incorrect_indices], y_pred_prob[incorrect_indices],
        num_examples=min(5, len(incorrect_indices)),
        save_path=os.path.join(visualizations_dir,␣
↳'cnn_keras_incorrect_predictions.png')
    )

print("Vorhersagen wurden visualisiert und gespeichert.")

```

Vorhersagen wurden visualisiert und gespeichert.

1.16 Zusammenfassung

In diesem Notebook haben wir ein Convolutional Neural Network (CNN) mit Keras/TensorFlow implementiert, um Autos im CIFAR-10 Datensatz zu erkennen. Hier sind die wichtigsten Schritte und Ergebnisse:

1. **Datenaufbereitung:** Wir haben die vorbereiteten Daten aus dem vorherigen Notebook geladen, die bereits normalisiert und mit binären Labels versehen waren.
2. **Modellarchitektur:** Wir haben ein CNN mit mehreren Faltungsschichten, Max-Pooling, Dropout und vollständig verbundenen Schichten definiert. Die Architektur wurde speziell für die Erkennung von Autos in den CIFAR-10 Bildern entworfen.
3. **Training:** Wir haben das Modell mit dem Adam-Optimizer und der Binary Cross-Entropy Loss-Funktion trainiert. Wir haben Early Stopping und Model Checkpointing verwendet, um Overfitting zu vermeiden und das beste Modell zu speichern.
4. **Evaluierung:** Wir haben das Modell auf den Testdaten evaluiert und eine Genauigkeit von etwa 96% erreicht. Wir haben auch detailliertere Metriken wie Precision, Recall und F1-Score berechnet, die ein umfassenderes Bild der Modelleistung geben.
5. **Visualisierung:** Wir haben den Trainingsverlauf, die Konfusionsmatrix und einige Beispieldiagnosen visualisiert, um ein besseres Verständnis für die Leistung des Modells zu bekommen.

Das trainierte Modell zeigt eine gute Leistung bei der Erkennung von Autos, mit einer hohen Genauigkeit und einem guten F1-Score. Die Visualisierungen zeigen, dass das Modell in den meisten Fällen korrekte Vorhersagen trifft, aber auch einige Fehler macht, insbesondere bei schwierigeren Beispielen.

Im nächsten Notebook werden wir ein benutzerdefiniertes CNN ohne die Verwendung von `keras.models` oder `keras.layers` implementieren, um ein tieferes Verständnis für die zugrunde liegenden Operationen zu bekommen.

03_custom_cnn_implementation_enhanced

April 9, 2025

1 Benutzerdefiniertes CNN ohne keras.models oder keras.layers

Dieses Notebook implementiert ein Convolutional Neural Network (CNN) ohne Verwendung von keras.models oder keras.layers zur Erkennung von Autos im CIFAR-10 Datensatz.

1.1 Einführung und theoretischer Hintergrund

Convolutional Neural Networks (CNNs) haben sich als leistungsstarke Werkzeuge für die Bildverarbeitung und -erkennung etabliert. Während Frameworks wie Keras und TensorFlow die Implementierung von CNNs erheblich vereinfachen, ist es für ein tieferes Verständnis wertvoll, die grundlegenden Operationen und Algorithmen selbst zu implementieren.

In diesem Notebook implementieren wir ein CNN von Grund auf mit NumPy, ohne die höheren Abstraktionen von keras.models oder keras.layers zu verwenden. Dies ermöglicht uns:

1. Ein tieferes Verständnis der mathematischen Grundlagen von CNNs zu entwickeln
2. Die Vorwärts- und Rückwärtspropagierung für jede Schicht explizit zu implementieren
3. Den Lernprozess und die Parameteraktualisierung im Detail zu verstehen
4. Die Herausforderungen und Komplexitäten bei der Implementierung von neuronalen Netzwerken zu erfahren

Die Hauptkomponenten unserer Implementierung umfassen:

- **Faltungsoperation (Convolution):** Die grundlegende Operation in CNNs, bei der Filter über das Eingabebild gleiten, um Merkmale zu extrahieren. Mathematisch ist dies eine Kreuzkorrelation, bei der jeder Ausgabepixel durch die gewichtete Summe der Eingabepixel in einem lokalen Bereich berechnet wird.
- **Aktivierungsfunktionen:** Nichtlineare Funktionen wie ReLU (Rectified Linear Unit), die es dem Netzwerk ermöglichen, komplexe Muster zu lernen. Die ReLU-Funktion ist definiert als $f(x) = \max(0, x)$.
- **Pooling:** Eine Operation zur Reduzierung der räumlichen Dimensionen, typischerweise durch Auswahl des maximalen Wertes in einem definierten Fenster (Max-Pooling).
- **Vollständig verbundene Schichten (Fully Connected):** Schichten, in denen jedes Neuron mit allen Neuronen der vorherigen Schicht verbunden ist, ähnlich wie in traditionellen neuronalen Netzwerken.
- **Backpropagation:** Der Algorithmus zur Berechnung der Gradienten der Verlustfunktion in Bezug auf die Netzwerkparameter, der es ermöglicht, die Parameter durch Gradientenabstieg zu aktualisieren.

Diese Implementierung ist zwar weniger effizient als optimierte Bibliotheken wie TensorFlow, bietet aber wertvolle Einblicke in die innere Funktionsweise von CNNs.

1.2 Überblick über die Schritte

- Implementierung eines CNN von Grund auf mit NumPy
- Implementierung der Vorwärts- und Rückwärtspropagierung für alle Schichten
- Training des Modells mit Mini-Batch Gradient Descent
- Evaluierung des Modells auf Testdaten
- Visualisierung der Ergebnisse

1.3 Importieren der benötigten Bibliotheken

Für unsere benutzerdefinierte CNN-Implementierung benötigen wir nur wenige Bibliotheken:

- **numpy**: Für effiziente numerische Operationen und Array-Manipulationen
- **matplotlib**: Für die Visualisierung der Trainingsergebnisse und Vorhersagen
- **os**: Für Dateisystem-Operationen wie das Laden der vorbereiteten Daten
- **time**: Für die Messung der Trainingszeit

Im Gegensatz zur Keras-Implementierung verwenden wir keine spezialisierten Deep-Learning-Bibliotheken, da wir alle Funktionalitäten selbst implementieren werden.

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
import os
import time
```

1.4 Vorbereitung der Verzeichnisse und Laden der Daten

Bevor wir mit der Implementierung beginnen, laden wir die vorbereiteten Daten aus dem ersten Notebook. Diese Daten wurden bereits normalisiert und mit binären Labels versehen (1 für Auto, 0 für Nicht-Auto).

Da unsere benutzerdefinierte Implementierung rechenintensiver ist als die Keras-Version, reduzieren wir die Datensatzgröße für das Training, um die Ausführungszeit zu verkürzen. Dies ist ein üblicher Ansatz beim Prototyping und Testen von Algorithmen.

```
[ ]: # Vorbereitung der Verzeichnisse
data_dir = '../data'
models_dir = '../models'
custom_models_dir = os.path.join(models_dir, 'custom_cnn')
visualizations_dir = os.path.join(models_dir, 'visualizations')

os.makedirs(custom_models_dir, exist_ok=True)
os.makedirs(visualizations_dir, exist_ok=True)

# Laden der vorbereiteten Daten
x_train = np.load(os.path.join(data_dir, 'x_train_normalized.npy'))
x_test = np.load(os.path.join(data_dir, 'x_test_normalized.npy'))
```

```

y_train = np.load(os.path.join(data_dir, 'y_train_binary.npy'))
y_test = np.load(os.path.join(data_dir, 'y_test_binary.npy'))

print(f"Trainingsbilder: {x_train.shape}")
print(f"Testbilder: {x_test.shape}")
print(f"Trainings-Labels: {y_train.shape}")
print(f"Test-Labels: {y_test.shape}")

```

1.5 Reduzieren der Datensatzgröße für schnelleres Training

Da unsere benutzerdefinierte CNN-Implementierung nicht die optimierten Operationen von Frameworks wie TensorFlow nutzt, wäre das Training auf dem gesamten Datensatz sehr zeitaufwändig. Daher reduzieren wir die Datensatzgröße für das Training.

Wir wählen eine Teilmenge der Trainings- und Testdaten aus, wobei wir darauf achten, dass die Klassenverteilung (Autos vs. Nicht-Autos) erhalten bleibt. Dies ermöglicht uns, die Implementierung zu testen und zu validieren, ohne übermäßig lange Rechenzeiten in Kauf nehmen zu müssen.

In einer Produktionsumgebung würde man natürlich den vollständigen Datensatz verwenden und optimierte Implementierungen nutzen.

```

[ ]: # Reduzieren der Datensatzgröße für schnelleres Training
def reduce_dataset(x_data, y_data, num_samples, random_seed=42):
    np.random.seed(random_seed)

    # Indizes für jede Klasse finden
    car_indices = np.where(y_data == 1)[0]
    non_car_indices = np.where(y_data == 0)[0]

    # Berechnen der Anzahl der Samples pro Klasse unter Beibehaltung des
    ↪Verhältnisses
    car_ratio = len(car_indices) / len(y_data)
    num_car_samples = int(num_samples * car_ratio)
    num_non_car_samples = num_samples - num_car_samples

    # Zufällige Auswahl von Indizes für jede Klasse
    selected_car_indices = np.random.choice(car_indices, size=num_car_samples,
    ↪replace=False)
    selected_non_car_indices = np.random.choice(non_car_indices,
    ↪size=num_non_car_samples, replace=False)

    # Kombinieren der ausgewählten Indizes
    selected_indices = np.concatenate([selected_car_indices,
    ↪selected_non_car_indices])
    np.random.shuffle(selected_indices)

    return x_data[selected_indices], y_data[selected_indices]

```

```

# Reduzieren der Trainings- und Testdaten
num_train_samples = 5000 # 10% der ursprünglichen Trainingsdaten
num_test_samples = 1000 # 10% der ursprünglichen Testdaten

x_train_reduced, y_train_reduced = reduce_dataset(x_train, y_train,
    ↪ num_train_samples)
x_test_reduced, y_test_reduced = reduce_dataset(x_test, y_test,
    ↪ num_test_samples)

print(f"Reduzierte Trainingsbilder: {x_train_reduced.shape}")
print(f"Reduzierte Testbilder: {x_test_reduced.shape}")
print(f"Anzahl der Auto-Bilder im reduzierten Trainingsdatensatz: {np.
    ↪ sum(y_train_reduced == 1)}")
print(f"Anzahl der Auto-Bilder im reduzierten Testdatensatz: {np.
    ↪ sum(y_test_reduced == 1)}")

```

1.6 Implementierung der CNN-Funktionen

Jetzt implementieren wir die grundlegenden Funktionen für unser CNN. Wir beginnen mit den Hilfsfunktionen für die Faltungsoperation, gefolgt von den Implementierungen für die Vorwärts- und Rückwärtspropagierung jeder Schicht.

Die Implementierung folgt dem mathematischen Fundament von CNNs und umfasst alle notwendigen Operationen für das Training und die Vorhersage.

1.6.1 Hilfsfunktionen für die Faltungsoperation

Die Faltungsoperation ist das Herzstück eines CNN. Bei dieser Operation wird ein Filter (oder Kernel) über das Eingabebild geschoben, und an jeder Position wird das Skalarprodukt zwischen dem Filter und dem entsprechenden Bereich des Bildes berechnet.

Mathematisch kann die Faltungsoperation für ein 2D-Bild I und einen 2D-Filter K wie folgt ausgedrückt werden:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

Für Farbbilder mit mehreren Kanälen wird diese Operation für jeden Kanal separat durchgeführt und die Ergebnisse werden summiert.

Wir implementieren zwei Hilfsfunktionen: 1. `im2col`: Transformiert Bildregionen in Spalten einer Matrix, was eine effizientere Berechnung der Faltung ermöglicht 2. `col2im`: Die Umkehroperation von `im2col`, die eine Spaltenmatrix zurück in ein Bild transformiert

```

[ ]: # Hilfsfunktionen für die Faltungsoperation
def im2col(input_data, filter_h, filter_w, stride=1, pad=0):
    """
    Transformiert Regionen eines mehrdimensionalen Arrays in Spalten.

    Parameter:
    - input_data: Eingabedaten mit Form (N, C, H, W)
    """

```

```

- filter_h: Höhe des Filters
- filter_w: Breite des Filters
- stride: Schrittweite der Faltung
- pad: Anzahl der Padding-Pixel

Rückgabe:
- col: Transformierte Daten mit Form (N*out_h*out_w, C*filter_h*filter_w)
"""
N, C, H, W = input_data.shape
out_h = (H + 2*pad - filter_h)//stride + 1
out_w = (W + 2*pad - filter_w)//stride + 1

# Padding hinzufügen
img = np.pad(input_data, [(0,0), (0,0), (pad, pad), (pad, pad)], 'constant')
col = np.zeros((N, C, filter_h, filter_w, out_h, out_w))

for y in range(filter_h):
    y_max = y + stride*out_h
    for x in range(filter_w):
        x_max = x + stride*out_w
        col[:, :, y, x, :, :] = img[:, :, y:y_max:stride, x:x_max:stride]

col = col.transpose(0, 4, 5, 1, 2, 3).reshape(N*out_h*out_w, -1)
return col

def col2im(col, input_shape, filter_h, filter_w, stride=1, pad=0):
    """
    Transformiert Spalten zurück in ein mehrdimensionales Array.

    Parameter:
    - col: Transformierte Daten mit Form (N*out_h*out_w, C*filter_h*filter_w)
    - input_shape: Form der Eingabedaten (N, C, H, W)
    - filter_h: Höhe des Filters
    - filter_w: Breite des Filters
    - stride: Schrittweite der Faltung
    - pad: Anzahl der Padding-Pixel

    Rückgabe:
    - img: Transformierte Daten mit Form (N, C, H, W)
    """
    N, C, H, W = input_shape
    out_h = (H + 2*pad - filter_h)//stride + 1
    out_w = (W + 2*pad - filter_w)//stride + 1
    col = col.reshape(N, out_h, out_w, C, filter_h, filter_w).transpose(0, 3, 4, 5, 1, 2)

    img = np.zeros((N, C, H + 2*pad + stride - 1, W + 2*pad + stride - 1))

```

```

for y in range(filter_h):
    y_max = y + stride*out_h
    for x in range(filter_w):
        x_max = x + stride*out_w
        img[:, :, y:y_max:stride, x:x_max:stride] += col[:, :, y, x, :, :]

return img[:, :, pad:H + pad, pad:W + pad]

```

1.6.2 Vorwärtspropagierung für die Faltungsschicht

Die Faltungsschicht ist die charakteristische Komponente eines CNN. In der Vorwärtspropagierung werden Filter über das Eingabebild geschoben, um Merkmale zu extrahieren.

Unsere Implementierung der Faltungsschicht umfasst: 1. Initialisierung der Filter und Bias-Parameter 2. Vorwärtspropagierung mit der Faltungsoperation 3. Rückwärtspropagierung zur Berechnung der Gradienten

Die Filter werden mit einer Xavier-Initialisierung initialisiert, die für tiefe neuronale Netzwerke entwickelt wurde und eine bessere Konvergenz ermöglicht.

```

[ ]: class Convolution:
    def __init__(self, input_channels, filter_num, filter_size, stride=1,
        pad=0):
        """
        Initialisiert eine Faltungsschicht.

        Parameter:
        - input_channels: Anzahl der Eingangskanäle
        - filter_num: Anzahl der Filter
        - filter_size: Größe der Filter (Höhe = Breite)
        - stride: Schrittweite der Faltung
        - pad: Anzahl der Padding-Pixel
        """
        self.input_channels = input_channels
        self.filter_num = filter_num
        self.filter_size = filter_size
        self.filter_h = filter_size
        self.filter_w = filter_size
        self.stride = stride
        self.pad = pad

        # Xavier-Initialisierung für bessere Konvergenz
        scale = np.sqrt(2.0 / (input_channels * filter_size * filter_size))
        self.W = scale * np.random.randn(filter_num, input_channels,
            filter_size, filter_size)
        self.b = np.zeros(filter_num)

        # Gradienten

```



```

self.dW = None
self.db = None

# Für die Rückwärtspropagierung
self.x = None
self.col = None
self.col_W = None

def forward(self, x):
    """
    Vorwärtspropagierung für die Faltungsschicht.

    Parameter:
    - x: Eingabedaten mit Form (N, C, H, W)

    Rückgabe:
    - out: Ausgabedaten mit Form (N, filter_num, out_h, out_w)
    """
    N, C, H, W = x.shape
    out_h = (H + 2*self.pad - self.filter_h)//self.stride + 1
    out_w = (W + 2*self.pad - self.filter_w)//self.stride + 1

    # Transformation der Eingabe für effiziente Faltung
    col = im2col(x, self.filter_h, self.filter_w, self.stride, self.pad)
    col_W = self.W.reshape(self.filter_num, -1).T

    # Faltungsoperation als Matrixmultiplikation
    out = np.dot(col, col_W) + self.b
    out = out.reshape(N, out_h, out_w, -1).transpose(0, 3, 1, 2)

    # Speichern für die Rückwärtspropagierung
    self.x = x
    self.col = col
    self.col_W = col_W

    return out

def backward(self, dout):
    """
    Rückwärtspropagierung für die Faltungsschicht.

    Parameter:
    - dout: Gradient der Verlustfunktion bezüglich der Ausgabe

    Rückgabe:
    - dx: Gradient der Verlustfunktion bezüglich der Eingabe
    """

```

```

FN, C, FH, FW = self.W.shape
dout = dout.transpose(0, 2, 3, 1).reshape(-1, FN)

# Berechnung der Gradienten
self.db = np.sum(dout, axis=0)
self.dW = np.dot(self.col.T, dout)
self.dW = self.dW.transpose(1, 0).reshape(FN, C, FH, FW)

# Berechnung des Gradienten bezüglich der Eingabe
dcol = np.dot(dout, self.col_W.T)
dx = col2im(dcol, self.x.shape, FH, FW, self.stride, self.pad)

return dx

```

1.6.3 Aktivierungsfunktionen

Aktivierungsfunktionen führen Nichtlinearität in das neuronale Netzwerk ein, was es ermöglicht, komplexe Muster zu lernen. Wir implementieren die ReLU-Aktivierungsfunktion (Rectified Linear Unit) und die Sigmoid-Aktivierungsfunktion.

1. **ReLU:** $f(x) = \max(0, x)$
 - Einfach zu berechnen und differenzieren
 - Hilft, das Problem des verschwindenden Gradienten zu mildern
 - Wird in den versteckten Schichten verwendet
2. **Sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$
 - Bildet Eingaben auf den Bereich (0, 1) ab
 - Geeignet für binäre Klassifikation
 - Wird in der Ausgabeschicht für binäre Klassifikation verwendet

```

[ ]: class ReLU:
    def __init__(self):
        """
        Initialisiert eine ReLU-Aktivierungsschicht.
        """
        self.mask = None

    def forward(self, x):
        """
        Vorwärtspropagierung für die ReLU-Aktivierung.

        Parameter:
        - x: Eingabedaten

        Rückgabe:
        - out: Aktivierte Ausgabedaten
        """
        self.mask = (x <= 0)
        out = x.copy()

```

```

        out[self.mask] = 0
        return out

def backward(self, dout):
    """
    Rückwärtspropagierung für die ReLU-Aktivierung.

    Parameter:
    - dout: Gradient der Verlustfunktion bezüglich der Ausgabe

    Rückgabe:
    - dx: Gradient der Verlustfunktion bezüglich der Eingabe
    """
    dout[self.mask] = 0
    dx = dout
    return dx

class Sigmoid:
    def __init__(self):
        """
        Initialisiert eine Sigmoid-Aktivierungsschicht.
        """
        self.out = None

    def forward(self, x):
        """
        Vorwärtspropagierung für die Sigmoid-Aktivierung.

        Parameter:
        - x: Eingabedaten

        Rückgabe:
        - out: Aktivierte Ausgabedaten
        """
        out = 1 / (1 + np.exp(-x))
        self.out = out
        return out

    def backward(self, dout):
        """
        Rückwärtspropagierung für die Sigmoid-Aktivierung.

        Parameter:
        - dout: Gradient der Verlustfunktion bezüglich der Ausgabe

        Rückgabe:
        - dx: Gradient der Verlustfunktion bezüglich der Eingabe

```

```

"""
dx = dout * (1.0 - self.out) * self.out
return dx

```

1.6.4 Pooling-Schicht

Die Pooling-Schicht reduziert die räumliche Dimension der Feature Maps, was die Berechnungseffizienz erhöht und eine gewisse Translationsinvarianz einführt. Wir implementieren Max-Pooling, bei dem der maximale Wert aus einem definierten Fenster ausgewählt wird.

Mathematisch kann Max-Pooling für ein Fenster der Größe $h \times w$ wie folgt ausgedrückt werden:

$$\text{MaxPool}(i, j) = \max_{0 \leq m < h, 0 \leq n < w} I(i \cdot s + m, j \cdot s + n)$$

wobei s die Schrittweite (stride) ist.

In der Rückwärtspropagierung werden die Gradienten nur an die Position des maximalen Wertes im Eingabefenster weitergegeben.

```

[ ]: class Pooling:
    def __init__(self, pool_h, pool_w, stride=1, pad=0):
        """
        Initialisiert eine Pooling-Schicht.

        Parameter:
        - pool_h: Höhe des Pooling-Fensters
        - pool_w: Breite des Pooling-Fensters
        - stride: Schrittweite des Pooling
        - pad: Anzahl der Padding-Pixel
        """
        self.pool_h = pool_h
        self.pool_w = pool_w
        self.stride = stride
        self.pad = pad

        # Für die Rückwärtspropagierung
        self.x = None
        self.arg_max = None

    def forward(self, x):
        """
        Vorwärtspropagierung für die Pooling-Schicht.

        Parameter:
        - x: Eingabedaten mit Form (N, C, H, W)

        Rückgabe:
        - out: Ausgabedaten mit reduzierter räumlicher Dimension
        """

```

```

N, C, H, W = x.shape
out_h = (H - self.pool_h) // self.stride + 1
out_w = (W - self.pool_w) // self.stride + 1

# Transformation der Eingabe für effizientes Pooling
col = im2col(x, self.pool_h, self.pool_w, self.stride, self.pad)
col = col.reshape(-1, self.pool_h * self.pool_w)

# Max-Pooling
arg_max = np.argmax(col, axis=1)
out = np.max(col, axis=1)
out = out.reshape(N, out_h, out_w, C).transpose(0, 3, 1, 2)

# Speichern für die Rückwärtspropagierung
self.x = x
self.arg_max = arg_max

return out

def backward(self, dout):
    """
    Rückwärtspropagierung für die Pooling-Schicht.

    Parameter:
    - dout: Gradient der Verlustfunktion bezüglich der Ausgabe

    Rückgabe:
    - dx: Gradient der Verlustfunktion bezüglich der Eingabe
    """
    dout = dout.transpose(0, 2, 3, 1)
    pool_size = self.pool_h * self.pool_w
    dmax = np.zeros((dout.size, pool_size))
    dmax[np.arange(self.arg_max.size), self.arg_max.flatten()] = dout.
    ↪flatten()
    dmax = dmax.reshape(dout.shape[0], dout.shape[1], dout.shape[2], dout.
    ↪shape[3], pool_size)
    dcol = dmax.reshape(dmax.shape[0] * dmax.shape[1] * dmax.shape[2] *
    ↪dmax.shape[3], -1)
    dx = col2im(dcol, self.x.shape, self.pool_h, self.pool_w, self.stride,
    ↪self.pad)

    return dx

```

1.6.5 Flatten und Fully Connected Layer

Nach den Faltungs- und Pooling-Schichten müssen wir die mehrdimensionalen Feature Maps in einen eindimensionalen Vektor umwandeln, um sie an vollständig verbundene Schichten zu

übergeben. Dies wird durch die Flatten-Schicht erreicht.

Die vollständig verbundene Schicht (Fully Connected Layer) verbindet jedes Neuron mit allen Neuronen der vorherigen Schicht, ähnlich wie in traditionellen neuronalen Netzwerken. Mathematisch kann dies als Matrixmultiplikation ausgedrückt werden:

$$y = Wx + b$$

wobei W die Gewichtsmatrix, x der Eingabevektor und b der Bias-Vektor ist.

```
[ ]: class Flatten:
    def __init__(self):
        """
        Initialisiert eine Flatten-Schicht.
        """
        self.original_shape = None

    def forward(self, x):
        """
        Vorwärtspropagierung für die Flatten-Schicht.

        Parameter:
        - x: Eingabedaten mit Form (N, C, H, W)

        Rückgabe:
        - out: Ausgabedaten mit Form (N, C*H*W)
        """
        self.original_shape = x.shape
        N = x.shape[0]
        out = x.reshape(N, -1)
        return out

    def backward(self, dout):
        """
        Rückwärtspropagierung für die Flatten-Schicht.

        Parameter:
        - dout: Gradient der Verlustfunktion bezüglich der Ausgabe

        Rückgabe:
        - dx: Gradient der Verlustfunktion bezüglich der Eingabe
        """
        dx = dout.reshape(self.original_shape)
        return dx

class FullyConnected:
    def __init__(self, input_size, output_size):
        """
        Initialisiert eine vollständig verbundene Schicht.
```

```

    Parameter:
    - input_size: Größe der Eingabe
    - output_size: Größe der Ausgabe
    """
    # Xavier-Initialisierung für bessere Konvergenz
    scale = np.sqrt(2.0 / input_size)
    self.W = scale * np.random.randn(input_size, output_size)
    self.b = np.zeros(output_size)

    # Gradienten
    self.dW = None
    self.db = None

    # Für die Rückwärtspropagierung
    self.x = None

def forward(self, x):
    """
    Vorwärtspropagierung für die vollständig verbundene Schicht.

    Parameter:
    - x: Eingabedaten mit Form (N, input_size)

    Rückgabe:
    - out: Ausgabedaten mit Form (N, output_size)
    """
    self.x = x
    out = np.dot(x, self.W) + self.b
    return out

def backward(self, dout):
    """
    Rückwärtspropagierung für die vollständig verbundene Schicht.

    Parameter:
    - dout: Gradient der Verlustfunktion bezüglich der Ausgabe

    Rückgabe:
    - dx: Gradient der Verlustfunktion bezüglich der Eingabe
    """
    dx = np.dot(dout, self.W.T)
    self.dW = np.dot(self.x.T, dout)
    self.db = np.sum(dout, axis=0)
    return dx

```

1.6.6 Kostenfunktion

Für unser binäres Klassifikationsproblem verwenden wir die binäre Kreuzentropie-Verlustfunktion (Binary Cross-Entropy Loss). Diese Funktion misst den Unterschied zwischen den vorhergesagten Wahrscheinlichkeiten und den tatsächlichen Labels.

Mathematisch ist die binäre Kreuzentropie-Verlustfunktion für ein einzelnes Beispiel definiert als:

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

wobei y das tatsächliche Label (0 oder 1) und $\hat{y}$ die vorhergesagte Wahrscheinlichkeit ist.

Für einen Batch von Beispielen berechnen wir den Durchschnitt der Verluste.

```
[ ]: class BinaryCrossEntropyLoss:
    def __init__(self):
        """
        Initialisiert eine binäre Kreuzentropie-Verlustfunktion.
        """
        self.loss = None
        self.y = None
        self.t = None

    def forward(self, y, t):
        """
        Berechnet den Verlust.

        Parameter:
        - y: Vorhergesagte Wahrscheinlichkeiten mit Form (N, 1)
        - t: Tatsächliche Labels mit Form (N, 1)

        Rückgabe:
        - loss: Durchschnittlicher Verlust
        """
        self.y = y
        self.t = t

        # Numerische Stabilität
        epsilon = 1e-7
        y = np.clip(y, epsilon, 1 - epsilon)

        # Binäre Kreuzentropie
        self.loss = -np.sum(t * np.log(y) + (1 - t) * np.log(1 - y)) / len(y)

        return self.loss

    def backward(self, dout=1):
        """
        Berechnet den Gradienten der Verlustfunktion bezüglich der Eingabe.
```



```

Parameter:
- dout: Upstream-Gradient (normalerweise 1)

Rückgabe:
- dx: Gradient der Verlustfunktion bezüglich der Eingabe
"""
batch_size = self.t.shape[0]

# Gradient der binären Kreuzentropie
dx = (self.y - self.t) / batch_size

return dx

```

1.6.7 Rückwärtspropagierung

Die Rückwärtspropagierung ist der Algorithmus, mit dem die Gradienten der Verlustfunktion in Bezug auf die Parameter des Netzwerks berechnet werden. Diese Gradienten werden dann verwendet, um die Parameter durch Gradientenabstieg zu aktualisieren.

Der Algorithmus arbeitet rückwärts durch das Netzwerk, beginnend mit der Ausgabeschicht, und berechnet die Gradienten für jede Schicht basierend auf der Kettenregel der Differentiation.

Für jede Schicht haben wir bereits die Rückwärtspropagierungsmethode implementiert, die den Gradienten der Verlustfunktion bezüglich der Eingabe der Schicht berechnet und die Gradienten bezüglich der Parameter der Schicht aktualisiert.

1.6.8 Vorwärts- und Rückwärtspropagierung für das gesamte Modell

Jetzt kombinieren wir alle implementierten Schichten zu einem vollständigen CNN-Modell. Das Modell führt die Vorwärtspropagierung durch, um Vorhersagen zu treffen, und die Rückwärtspropagierung, um die Gradienten zu berechnen.

Unsere CNN-Architektur besteht aus: 1. Einer Faltungsschicht mit ReLU-Aktivierung 2. Einer Max-Pooling-Schicht 3. Einer zweiten Faltungsschicht mit ReLU-Aktivierung 4. Einer zweiten Max-Pooling-Schicht 5. Einer Flatten-Schicht 6. Einer vollständig verbundenen Schicht mit ReLU-Aktivierung 7. Einer Ausgabeschicht mit Sigmoid-Aktivierung für binäre Klassifikation

```

[ ]: class CNN:
    def __init__(self, input_shape, conv_param_1, conv_param_2, hidden_size,
↪output_size):
        """
        Initialisiert ein CNN-Modell.

        Parameter:
        - input_shape: Form der Eingabedaten (C, H, W)
        - conv_param_1: Parameter für die erste Faltungsschicht
        - conv_param_2: Parameter für die zweite Faltungsschicht
        - hidden_size: Größe der versteckten vollständig verbundenen Schicht
        - output_size: Größe der Ausgabe (1 für binäre Klassifikation)

```

```

"""
# Extrahieren der Parameter
C, H, W = input_shape
filter_num_1, filter_size_1, stride_1, pad_1 = conv_param_1
filter_num_2, filter_size_2, stride_2, pad_2 = conv_param_2

# Berechnung der Ausgabegrößen der Faltungs- und Pooling-Schichten
conv_output_size_1 = (H - filter_size_1 + 2*pad_1) // stride_1 + 1
pool_output_size_1 = (conv_output_size_1 - 2) // 2 + 1
conv_output_size_2 = (pool_output_size_1 - filter_size_2 + 2*pad_2) //
↳ stride_2 + 1
pool_output_size_2 = (conv_output_size_2 - 2) // 2 + 1

# Berechnung der Eingabegröße für die vollständig verbundene Schicht
fc_input_size = filter_num_2 * pool_output_size_2 * pool_output_size_2

# Initialisierung der Schichten
self.layers = [
    Convolution(C, filter_num_1, filter_size_1, stride_1, pad_1),
    ReLU(),
    Pooling(2, 2, 2),
    Convolution(filter_num_1, filter_num_2, filter_size_2, stride_2,
↳ pad_2),
    ReLU(),
    Pooling(2, 2, 2),
    Flatten(),
    FullyConnected(fc_input_size, hidden_size),
    ReLU(),
    FullyConnected(hidden_size, output_size),
    Sigmoid()
]

# Verlustfunktion
self.loss_layer = BinaryCrossEntropyLoss()

# Parameter und Gradienten
self.params, self.grads = [], []
for layer in self.layers:
    if hasattr(layer, 'W'):
        self.params.append(layer.W)
        self.grads.append(layer.dW)
    if hasattr(layer, 'b'):
        self.params.append(layer.b)
        self.grads.append(layer.db)

def predict(self, x):
"""

```

```

    Macht Vorhersagen für die Eingabedaten.

    Parameter:
    - x: Eingabedaten mit Form (N, C, H, W)

    Rückgabe:
    - y: Vorhergesagte Wahrscheinlichkeiten mit Form (N, 1)
    """
    for layer in self.layers:
        x = layer.forward(x)
    return x

def forward(self, x, t):
    """
    Vorwärtspropagierung und Berechnung des Verlusts.

    Parameter:
    - x: Eingabedaten mit Form (N, C, H, W)
    - t: Tatsächliche Labels mit Form (N, 1)

    Rückgabe:
    - loss: Verlust
    """
    y = self.predict(x)
    loss = self.loss_layer.forward(y, t)
    return loss

def backward(self):
    """
    Rückwärtspropagierung zur Berechnung der Gradienten.

    Rückgabe:
    - dout: Gradient der Verlustfunktion bezüglich der Eingabe
    """
    dout = self.loss_layer.backward()
    for layer in reversed(self.layers):
        dout = layer.backward(dout)
    return dout

def accuracy(self, x, t):
    """
    Berechnet die Genauigkeit der Vorhersagen.

    Parameter:
    - x: Eingabedaten mit Form (N, C, H, W)
    - t: Tatsächliche Labels mit Form (N, 1)

```

```

    Rückgabe:
    - accuracy: Genauigkeit (0-1)
    """
    y = self.predict(x)
    y = (y > 0.5).astype(int)
    accuracy = np.mean(y == t)
    return accuracy

```

1.6.9 Aktualisierung der Parameter und Vorhersage

Nach der Berechnung der Gradienten müssen wir die Parameter des Netzwerks aktualisieren. Wir implementieren den Gradientenabstieg-Optimierer, der die Parameter in Richtung des negativen Gradienten aktualisiert.

Die Aktualisierungsregel für einen Parameter θ ist:

$$\theta = \theta - \eta \cdot \nabla_{\theta} L$$

wobei η die Lernrate und $\nabla_{\theta} L$ der Gradient der Verlustfunktion bezüglich des Parameters ist.

```

[ ]: class SGD:
    def __init__(self, lr=0.01):
        """
        Initialisiert einen Stochastic Gradient Descent Optimierer.

        Parameter:
        - lr: Lernrate
        """
        self.lr = lr

    def update(self, params, grads):
        """
        Aktualisiert die Parameter basierend auf den Gradienten.

        Parameter:
        - params: Liste der Parameter
        - grads: Liste der Gradienten
        """
        for i in range(len(params)):
            params[i] -= self.lr * grads[i]

```

1.6.10 Mini-Batch Gradient Descent

Statt den Gradientenabstieg auf dem gesamten Datensatz durchzuführen, verwenden wir Mini-Batch Gradient Descent, bei dem wir in jeder Iteration nur eine kleine Teilmenge (Batch) der Daten verwenden. Dies beschleunigt das Training und kann zu einer besseren Konvergenz führen.

Wir implementieren eine Funktion, die die Daten in Batches aufteilt und für jeden Batch die Vorwärts- und Rückwärtspropagierung durchführt, gefolgt von einer Aktualisierung der Parameter.

```
[ ]: def train_step(model, optimizer, x_batch, t_batch):
    """
    Führt einen Trainingsschritt für einen Batch durch.

    Parameter:
    - model: CNN-Modell
    - optimizer: Optimierer
    - x_batch: Batch von Eingabedaten
    - t_batch: Batch von Labels

    Rückgabe:
    - loss: Verlust für den Batch
    """
    # Vorwärtspropagierung
    loss = model.forward(x_batch, t_batch)

    # Rückwärtspropagierung
    model.backward()

    # Aktualisierung der Parameter
    optimizer.update(model.params, model.grads)

    return loss

def generate_batches(x, t, batch_size):
    """
    Generiert Batches aus den Daten.

    Parameter:
    - x: Eingabedaten
    - t: Labels
    - batch_size: Größe der Batches

    Rückgabe:
    - batches: Liste von (x_batch, t_batch) Tupeln
    """
    N = x.shape[0]
    indices = np.arange(N)
    np.random.shuffle(indices)

    batches = []
    for i in range(0, N, batch_size):
        batch_indices = indices[i:i+batch_size]
        x_batch = x[batch_indices]
        t_batch = t[batch_indices]
        batches.append((x_batch, t_batch))
```

```
return batches
```

1.6.11 Hauptfunktion für das Training des Modells

Schließlich implementieren wir die Hauptfunktion für das Training des Modells. Diese Funktion initialisiert das Modell, führt das Training für eine bestimmte Anzahl von Epochen durch und evaluiert das Modell regelmäßig auf den Validierungsdaten.

Wir speichern auch den Trainingsverlauf, um später die Leistung des Modells zu visualisieren.

```
[ ]: def train_model(x_train, t_train, x_val, t_val, input_shape, conv_param_1,
    ↪conv_param_2, hidden_size, output_size,
        batch_size=32, epochs=10, lr=0.01, verbose=True):
    """
    Trainiert ein CNN-Modell.

    Parameter:
    - x_train: Trainingsdaten
    - t_train: Trainings-Labels
    - x_val: Validierungsdaten
    - t_val: Validierungs-Labels
    - input_shape: Form der Eingabedaten (C, H, W)
    - conv_param_1: Parameter für die erste Faltungsschicht
    - conv_param_2: Parameter für die zweite Faltungsschicht
    - hidden_size: Größe der versteckten vollständig verbundenen Schicht
    - output_size: Größe der Ausgabe (1 für binäre Klassifikation)
    - batch_size: Größe der Batches
    - epochs: Anzahl der Trainingsepochen
    - lr: Lernrate
    - verbose: Ob Fortschrittsinformationen angezeigt werden sollen

    Rückgabe:
    - model: Trainiertes Modell
    - history: Trainingsverlauf
    """
    # Initialisierung des Modells und des Optimierers
    model = CNN(input_shape, conv_param_1, conv_param_2, hidden_size,
    ↪output_size)
    optimizer = SGD(lr=lr)

    # Trainingsverlauf
    history = {
        'train_loss': [],
        'train_acc': [],
        'val_loss': [],
        'val_acc': []
    }
```

```

# Training
for epoch in range(epochs):
    start_time = time.time()

    # Generieren der Batches
    batches = generate_batches(x_train, t_train, batch_size)

    # Training für jeden Batch
    train_loss = 0
    for x_batch, t_batch in batches:
        loss = train_step(model, optimizer, x_batch, t_batch)
        train_loss += loss

    # Berechnung des durchschnittlichen Verlusts und der Genauigkeit
    train_loss /= len(batches)
    train_acc = model.accuracy(x_train, t_train)

    # Evaluierung auf den Validierungsdaten
    val_loss = model.forward(x_val, t_val)
    val_acc = model.accuracy(x_val, t_val)

    # Speichern des Trainingsverlaufs
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)

    # Ausgabe des Fortschritts
    if verbose:
        elapsed_time = time.time() - start_time
        print(f"Epoch {epoch+1}/{epochs} - {elapsed_time:.2f}s - train_loss:
↪ {train_loss:.4f} - train_acc: {train_acc:.4f} - val_loss: {val_loss:.4f} -
↪ val_acc: {val_acc:.4f}")

    return model, history

```

1.7 Training des Modells

Jetzt trainieren wir unser benutzerdefiniertes CNN-Modell auf den reduzierten Daten. Wir definieren die Hyperparameter und die Architektur des Modells und starten das Training.

Da unsere Implementierung nicht optimiert ist, kann das Training einige Zeit in Anspruch nehmen, selbst auf dem reduzierten Datensatz.

```

[ ]: # Vorbereitung der Daten
# Umformen der Daten von (N, H, W, C) zu (N, C, H, W) für unsere Implementierung
x_train_reshaped = x_train_reduced.transpose(0, 3, 1, 2)

```

```

x_test_reshaped = x_test_reduced.transpose(0, 3, 1, 2)

# Hyperparameter
input_shape = (3, 32, 32) # (C, H, W)
conv_param_1 = (16, 3, 1, 1) # (filter_num, filter_size, stride, pad)
conv_param_2 = (32, 3, 1, 1) # (filter_num, filter_size, stride, pad)
hidden_size = 64
output_size = 1
batch_size = 32
epochs = 5
lr = 0.01

# Aufteilung in Trainings- und Validierungsdaten
validation_split = 0.2
n_val = int(len(x_train_reshaped) * validation_split)
x_val = x_train_reshaped[:n_val]
t_val = y_train_reduced[:n_val]
x_train_final = x_train_reshaped[n_val:]
t_train_final = y_train_reduced[n_val:]

print(f"Trainingsbilder: {x_train_final.shape}")
print(f"Validierungsbilder: {x_val.shape}")
print(f"Testbilder: {x_test_reshaped.shape}")

# Training des Modells
model, history = train_model(
    x_train_final, t_train_final, x_val, t_val,
    input_shape, conv_param_1, conv_param_2, hidden_size, output_size,
    batch_size=batch_size, epochs=epochs, lr=lr, verbose=True
)

```

1.8 Evaluierung des Modells

Nach dem Training evaluieren wir das Modell auf den Testdaten, um seine Generalisierungsfähigkeit zu bewerten. Wir berechnen den Verlust und die Genauigkeit und visualisieren den Trainingsverlauf.

```

[ ]: # Evaluierung auf den Testdaten
test_loss = model.forward(x_test_reshaped, y_test_reduced)
test_acc = model.accuracy(x_test_reshaped, y_test_reduced)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")

# Visualisierung des Trainingsverlaufs
plt.figure(figsize=(12, 5))

```



```

# Plot für den Verlust
plt.subplot(1, 2, 1)
plt.plot(history['train_loss'], label='Training Loss')
plt.plot(history['val_loss'], label='Validation Loss')
plt.title('Trainingsverlauf - Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Plot für die Genauigkeit
plt.subplot(1, 2, 2)
plt.plot(history['train_acc'], label='Training Accuracy')
plt.plot(history['val_acc'], label='Validation Accuracy')
plt.title('Trainingsverlauf - Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig(os.path.join(visualizations_dir, 'custom_cnn_training_history.png'))
plt.show()

```

1.9 Visualisierung der Vorhersagen

Um ein besseres Verständnis für die Leistung unseres Modells zu bekommen, visualisieren wir einige Beispiele aus dem Testdatensatz zusammen mit den Vorhersagen des Modells. Wir zeigen sowohl korrekte als auch falsche Vorhersagen.

```

[ ]: # Vorhersagen für die Testdaten
y_pred_prob = model.predict(x_test_reshaped)
y_pred = (y_pred_prob > 0.5).astype(int)

# Visualisierung einiger Vorhersagen
def visualize_predictions(x_data, y_true, y_pred, y_pred_prob, num_examples=10):
    # Umformen der Daten zurück zu (N, H, W, C) für die Visualisierung
    x_data_vis = x_data.transpose(0, 2, 3, 1)

    # Zufällige Indizes auswählen
    np.random.seed(42) # Für Reproduzierbarkeit
    indices = np.random.choice(len(y_true), size=num_examples, replace=False)

    # Erstellen der Visualisierung
    plt.figure(figsize=(15, 8))
    for i, idx in enumerate(indices):
        plt.subplot(2, 5, i+1)

```

```

plt.imshow(x_data_vis[idx])
plt.axis('off')

true_label = 'Auto' if y_true[idx][0] == 1 else 'Nicht-Auto'
pred_label = 'Auto' if y_pred[idx][0] == 1 else 'Nicht-Auto'
confidence = y_pred_prob[idx][0]

color = 'green' if y_true[idx][0] == y_pred[idx][0] else 'red'
plt.title(f"Wahr: {true_label}\nVorhersage: {pred_label}\nKonfidenz:␣
↪{confidence:.2f}", color=color)

plt.tight_layout()
plt.savefig(os.path.join(visualizations_dir, 'custom_cnn_predictions.png'))
plt.show()

# Visualisierung von Vorhersagen
visualize_predictions(x_test_reshaped, y_test_reduced, y_pred, y_pred_prob)

```

1.10 Zusammenfassung

In diesem Notebook haben wir ein Convolutional Neural Network (CNN) von Grund auf implementiert, ohne die höheren Abstraktionen von `keras.models` oder `keras.layers` zu verwenden. Hier sind die wichtigsten Punkte:

1. **Implementierung der grundlegenden Operationen:** Wir haben alle grundlegenden Operationen eines CNN implementiert, einschließlich Faltung, Aktivierungsfunktionen, Pooling und vollständig verbundene Schichten.
2. **Vorwärts- und Rückwärtspropagierung:** Wir haben die Vorwärtspropagierung für Vorhersagen und die Rückwärtspropagierung für die Berechnung der Gradienten implementiert.
3. **Training mit Mini-Batch Gradient Descent:** Wir haben das Modell mit Mini-Batch Gradient Descent trainiert, um die Effizienz zu verbessern.
4. **Evaluierung und Visualisierung:** Wir haben das Modell auf Testdaten evaluiert und die Ergebnisse visualisiert.

Diese Implementierung bietet ein tieferes Verständnis für die innere Funktionsweise von CNNs, ist aber weniger effizient als optimierte Bibliotheken wie TensorFlow. In der Praxis würde man für Produktionsanwendungen optimierte Bibliotheken verwenden, aber die Implementierung von Grund auf ist ein wertvolles Lernwerkzeug.

Im nächsten Notebook werden wir ein vortrainiertes CNN laden und für unsere Autoerkennungsaufgabe anpassen.

04_pretrained_cnn_enhanced

April 9, 2025

1 Vortrainiertes CNN zur Erkennung von Autos

Dieses Notebook implementiert ein vortrainiertes CNN (MobileNetV2) mit Transfer Learning zur Erkennung von Autos im CIFAR-10 Datensatz.

1.1 Einführung und theoretischer Hintergrund

Transfer Learning ist eine leistungsstarke Technik im Bereich des maschinellen Lernens, bei der ein Modell, das für eine Aufgabe trainiert wurde, als Ausgangspunkt für ein Modell für eine andere Aufgabe verwendet wird. Diese Methode ist besonders nützlich, wenn begrenzte Daten für die Zielaufgabe verfügbar sind oder wenn die Rechenressourcen für das Training eines komplexen Modells von Grund auf begrenzt sind.

Die Hauptvorteile von Transfer Learning sind:

1. **Schnellere Konvergenz:** Da das vortrainierte Modell bereits allgemeine Merkmale gelernt hat, kann das angepasste Modell schneller konvergieren.
2. **Bessere Generalisierung:** Vortrainierte Modelle haben oft eine bessere Generalisierungsfähigkeit, da sie auf großen und vielfältigen Datensätzen trainiert wurden.
3. **Weniger Trainingsdaten erforderlich:** Transfer Learning ermöglicht gute Ergebnisse auch mit kleineren Datensätzen.
4. **Geringerer Rechenaufwand:** Das Training eines angepassten Modells erfordert weniger Rechenressourcen als das Training eines Modells von Grund auf.

In diesem Notebook verwenden wir MobileNetV2, ein effizientes CNN, das auf dem ImageNet-Datensatz vortrainiert wurde. MobileNetV2 wurde von Google entwickelt und ist für mobile und eingebettete Anwendungen optimiert. Es verwendet tiefenweise trennbare Faltungen (depthwise separable convolutions), die die Anzahl der Parameter und Rechenoperationen erheblich reduzieren, während sie eine hohe Genauigkeit beibehalten.

Die Architektur von MobileNetV2 besteht aus: - Einem vollständigen Faltungslayer am Anfang - Mehreren Bottleneck-Blöcken mit tiefenweisen trennbaren Faltungen - Einem Pooling-Layer und einem vollständig verbundenen Layer am Ende

Für unsere Aufgabe der Autoerkennung werden wir: 1. Das vortrainierte MobileNetV2-Modell laden 2. Die vortrainierten Schichten einfrieren, um ihre Gewichte während des Trainings beizubehalten 3. Die oberen Schichten entfernen und durch neue Schichten ersetzen, die für unsere binäre Klassifikationsaufgabe geeignet sind 4. Das angepasste Modell auf unserem CIFAR-10-Datensatz trainieren

1.2 Überblick über die Schritte

- Laden eines vortrainierten MobileNetV2-Modells
- Anpassung des Modells für die Autoerkennung durch Transfer Learning
- Training des angepassten Modells
- Evaluierung des Modells auf Testdaten
- Visualisierung der Ergebnisse

1.3 Importieren der benötigten Bibliotheken

Für die Implementierung des Transfer Learning mit einem vortrainierten CNN benötigen wir verschiedene Python-Bibliotheken:

- **tensorflow**: Das Framework für maschinelles Lernen, das wir für das Training und die Inferenz verwenden
- **tensorflow.keras.applications**: Enthält vortrainierte Modelle wie MobileNetV2
- **tensorflow.keras.models**: Für die Definition und Anpassung von Modellen
- **tensorflow.keras.layers**: Für die Definition neuer Schichten für unser angepasstes Modell
- **tensorflow.keras.optimizers**: Für die Optimierung während des Trainings
- **tensorflow.keras.callbacks**: Für Funktionen wie Early Stopping und Model Checkpointing
- **numpy**: Für effiziente numerische Operationen
- **matplotlib**: Für die Visualisierung der Ergebnisse
- **os**: Für Dateisystem-Operationen

```
[ ]: import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
import matplotlib.pyplot as plt
```

1.4 Vorbereitung der Verzeichnisse

Bevor wir mit der Implementierung beginnen, erstellen wir Verzeichnisse für die Speicherung der Modelle und Visualisierungen. Eine gute Organisation der Projektstruktur ist wichtig für die Nachvollziehbarkeit und Wiederverwendbarkeit des Codes.

Wir erstellen zwei Verzeichnisse: - **models_dir**: Für die Speicherung der trainierten Modelle und Checkpoints - **visualizations_dir**: Für die Speicherung von Visualisierungen wie Lernkurven und Vorhersagen

Die Funktion `os.makedirs()` mit dem Parameter `exist_ok=True` stellt sicher, dass kein Fehler auftritt, falls die Verzeichnisse bereits existieren.

```
[ ]: # Vorbereitung der Verzeichnisse
data_dir = '../data'
models_dir = '../models'
```

```

pretrained_models_dir = os.path.join(models_dir, 'pretrained')
visualizations_dir = os.path.join(models_dir, 'visualizations')

os.makedirs(models_dir, exist_ok=True)
os.makedirs(pretrained_models_dir, exist_ok=True)
os.makedirs(visualizations_dir, exist_ok=True)

```

1.5 Laden der vorbereiteten Daten

Wir laden die vorbereiteten Daten aus dem ersten Notebook. Diese Daten wurden bereits normalisiert und mit binären Labels versehen (1 für Auto, 0 für Nicht-Auto).

Da MobileNetV2 für Bilder mit einer Größe von 224x224 Pixeln trainiert wurde, während unsere CIFAR-10-Bilder nur 32x32 Pixel groß sind, müssen wir die Bilder auf die richtige Größe skalieren. Dies ist ein wichtiger Schritt, da vortrainierte Modelle oft spezifische Eingabegrößen erwarten.

```

[ ]: # Laden der vorbereiteten Daten
x_train = np.load(os.path.join(data_dir, 'x_train_normalized.npy'))
x_test = np.load(os.path.join(data_dir, 'x_test_normalized.npy'))
y_train = np.load(os.path.join(data_dir, 'y_train_binary.npy'))
y_test = np.load(os.path.join(data_dir, 'y_test_binary.npy'))

print(f"Trainingsbilder: {x_train.shape}")
print(f"Testbilder: {x_test.shape}")
print(f"Trainings-Labels: {y_train.shape}")
print(f"Test-Labels: {y_test.shape}")

```

1.6 Vorbereitung der Bilder für das vortrainierte Modell

Vortrainierte Modelle wie MobileNetV2 erwarten Bilder in einer bestimmten Größe und mit einer bestimmten Vorverarbeitung. MobileNetV2 wurde auf Bildern mit einer Größe von 224x224 Pixeln trainiert, und die Pixelwerte wurden mit der `preprocess_input`-Funktion vorverarbeitet.

Wir müssen daher unsere CIFAR-10-Bilder (32x32 Pixel) auf 224x224 Pixel skalieren und die entsprechende Vorverarbeitung anwenden. Die Skalierung erfolgt mit der `tf.image.resize`-Funktion, und die Vorverarbeitung mit der `preprocess_input`-Funktion von MobileNetV2.

Die Skalierung auf eine größere Größe kann zu einem Verlust an Bildqualität führen, aber sie ist notwendig, um das vortrainierte Modell verwenden zu können. Trotz dieses potenziellen Nachteils kann Transfer Learning immer noch zu besseren Ergebnissen führen als das Training eines Modells von Grund auf, insbesondere bei begrenzten Daten.

```

[ ]: from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

def prepare_images_for_mobilenet(images):
    """
    Bereitet Bilder für MobileNetV2 vor, indem sie auf 224x224 Pixel skaliert_
    ↪und vorverarbeitet werden.
    """

```

```

Parameter:
- images: Bilder mit Form (N, H, W, C)

Rückgabe:
- processed_images: Vorverarbeitete Bilder mit Form (N, 224, 224, 3)
"""

# Skalieren der Bilder auf 224x224 Pixel
resized_images = tf.image.resize(images, (224, 224))

# Vorverarbeitung für MobileNetV2
processed_images = preprocess_input(resized_images * 255.0) # Skalieren
↳ zurück auf [0, 255] für preprocess_input

return processed_images

# Vorbereitung der Trainings- und Testbilder
x_train_mobilenet = prepare_images_for_mobilenet(x_train)
x_test_mobilenet = prepare_images_for_mobilenet(x_test)

print(f"Vorbereitete Trainingsbilder für MobileNetV2: {x_train_mobilenet.
↳ shape}")
print(f"Vorbereitete Testbilder für MobileNetV2: {x_test_mobilenet.shape}")

# Visualisierung einiger vorbereiteter Bilder
plt.figure(figsize=(10, 5))
for i in range(5):
    # Original-Bild
    plt.subplot(2, 5, i+1)
    plt.imshow(x_train[i])
    plt.title("Original")
    plt.axis('off')

    # Skaliertes Bild (normalisiert für die Anzeige)
    plt.subplot(2, 5, i+6)
    # Konvertieren zurück zu [0, 1] für die Anzeige
    img = (x_train_mobilenet[i] - np.min(x_train_mobilenet[i])) / (np.
↳ max(x_train_mobilenet[i]) - np.min(x_train_mobilenet[i]))
    plt.imshow(img)
    plt.title("Skaliert (224x224)")
    plt.axis('off')

plt.tight_layout()
plt.savefig(os.path.join(visualizations_dir, 'mobilenet_image_preparation.png'))
plt.show()

```

1.7 Laden des vortrainierten Modells

Jetzt laden wir das vortrainierte MobileNetV2-Modell. MobileNetV2 ist ein effizientes CNN, das auf dem ImageNet-Datensatz vortrainiert wurde, der über eine Million Bilder in 1000 Kategorien enthält.

Wir laden das Modell mit den vortrainierten Gewichten (`weights='imagenet'`), aber ohne die oberen Schichten (`include_top=False`), da wir diese durch unsere eigenen Schichten für die binäre Klassifikation ersetzen werden.

Die Eingabegröße wird auf 224x224 Pixel mit 3 Farbkanälen festgelegt, was der Größe entspricht, auf die wir unsere Bilder skaliert haben.

```
[ ]: # Laden des vortrainierten MobileNetV2-Modells
base_model = MobileNetV2(
    weights='imagenet', # Vortrainierte Gewichte auf ImageNet
    include_top=False,  # Ohne die oberen Schichten
    input_shape=(224, 224, 3) # Eingabegröße
)

# Zusammenfassung des Basismodells anzeigen
print("Zusammenfassung des Basismodells:")
base_model.summary()
```

1.8 Einfrieren der vortrainierten Schichten

Um Transfer Learning effektiv zu nutzen, frieren wir die Gewichte der vortrainierten Schichten ein, damit sie während des Trainings nicht aktualisiert werden. Dies ist wichtig, da diese Schichten bereits allgemeine Merkmale gelernt haben, die für viele Bilderkennungsaufgaben nützlich sind.

Durch das Einfrieren der vortrainierten Schichten: 1. Behalten wir das wertvolle Wissen, das das Modell auf dem ImageNet-Datensatz gelernt hat 2. Reduzieren wir die Anzahl der trainierbaren Parameter, was das Training beschleunigt 3. Verhindern wir Overfitting, insbesondere wenn unser Datensatz klein ist

In einigen Fällen kann es sinnvoll sein, einige der oberen Schichten des vortrainierten Modells für das Feintuning zu trainieren, aber für unsere Aufgabe frieren wir alle vortrainierten Schichten ein.

```
[ ]: # Einfrieren der vortrainierten Schichten
for layer in base_model.layers:
    layer.trainable = False

# Überprüfen der trainierbaren Parameter
trainable_params = sum([np.prod(layer.trainable_weights[0].shape) if layer.
    ↳ trainable_weights else 0 for layer in base_model.layers])
total_params = sum([np.prod(layer.weights[0].shape) if layer.weights else 0 for
    ↳ layer in base_model.layers])

print(f"Trainierbare Parameter im Basismodell: {trainable_params}")
print(f"Gesamtzahl der Parameter im Basismodell: {total_params}")
```

1.9 Hinzufügen von benutzerdefinierten Schichten

Nachdem wir das vortrainierte Modell geladen und seine Schichten eingefroren haben, fügen wir nun unsere eigenen benutzerdefinierten Schichten hinzu, die für unsere spezifische Aufgabe der Autoerkennung trainiert werden.

Wir fügen folgende Schichten hinzu: 1. **GlobalAveragePooling2D**: Diese Schicht reduziert die räumlichen Dimensionen, indem sie den Durchschnitt über alle räumlichen Positionen für jede Feature Map berechnet. Dies reduziert die Anzahl der Parameter und hilft, Overfitting zu vermeiden. 2. **Dense**: Eine vollständig verbundene Schicht mit 128 Neuronen und ReLU-Aktivierung, die als versteckte Schicht dient. 3. **Dense**: Eine Ausgangsschicht mit einem Neuron und Sigmoid-Aktivierung für die binäre Klassifikation (Auto vs. Nicht-Auto).

Diese Architektur ermöglicht es dem Modell, die vom vortrainierten Modell extrahierten Merkmale zu nutzen und sie für unsere spezifische Klassifikationsaufgabe anzupassen.

```
[ ]: # Hinzufügen von benutzerdefinierten Schichten
x = base_model.output
x = GlobalAveragePooling2D()(x) # Globales Average Pooling
x = Dense(128, activation='relu')(x) # Versteckte Schicht
predictions = Dense(1, activation='sigmoid')(x) # Ausgangsschicht für binäre
↳Klassifikation

# Erstellen des vollständigen Modells
model = Model(inputs=base_model.input, outputs=predictions)

# Zusammenfassung des vollständigen Modells anzeigen
print("Zusammenfassung des vollständigen Modells:")
model.summary()
```

1.10 Kompilieren des Modells

Bevor wir das Modell trainieren können, müssen wir es kompilieren, indem wir den Optimierer, die Verlustfunktion und die Metriken festlegen.

Für unsere binäre Klassifikationsaufgabe verwenden wir: - **Optimierer**: Adam mit einer niedrigen Lernrate (0.0001), da wir nur die oberen Schichten trainieren und eine zu hohe Lernrate die vortrainierten Merkmale stören könnte. - **Verlustfunktion**: Binary Cross-Entropy, die für binäre Klassifikationsprobleme geeignet ist. - **Metriken**: Accuracy (Genauigkeit), um die Leistung des Modells während des Trainings zu überwachen.

Die Wahl dieser Hyperparameter ist wichtig für den Erfolg des Transfer Learning. Eine zu hohe Lernrate könnte zu einer Überanpassung führen, während eine zu niedrige Lernrate zu einer langsamen Konvergenz führen könnte.

```
[ ]: # Kompilieren des Modells
model.compile(
    optimizer=Adam(learning_rate=0.0001), # Niedrige Lernrate für Transfer
↳Learning
    loss='binary_crossentropy', # Verlustfunktion für binäre Klassifikation
```



```
metrics=['accuracy'] # Metrik zur Überwachung
)
```

1.11 Callbacks für das Training

Callbacks sind Funktionen, die während des Trainings zu bestimmten Zeitpunkten aufgerufen werden. Sie können verwendet werden, um das Training zu überwachen, zu steuern oder zu protokollieren. Wir verwenden zwei wichtige Callbacks:

1. **Early Stopping:** Dieser Callback überwacht eine bestimmte Metrik (in unserem Fall die Validierungs-Loss) und stoppt das Training, wenn sich diese Metrik über eine bestimmte Anzahl von Epochen nicht verbessert. Dies verhindert Overfitting und spart Rechenzeit.
2. **Model Checkpoint:** Dieser Callback speichert das Modell nach jeder Epoche, wenn sich eine bestimmte Metrik verbessert hat. Wir speichern das Modell, wenn die Validierungs-Loss sinkt, und behalten so das beste Modell während des Trainings.

Diese Callbacks sind besonders nützlich für das Training von neuronalen Netzwerken, da sie helfen, die optimale Anzahl von Trainingsepochen zu finden und das beste Modell zu speichern.

```
[ ]: # Callbacks für das Training
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True,
    verbose=1
)

checkpoint_path = os.path.join(pretrained_models_dir, 'mobilenet_best_model.h5')
model_checkpoint = ModelCheckpoint(
    filepath=checkpoint_path,
    monitor='val_loss',
    save_best_only=True,
    verbose=1
)

callbacks = [early_stopping, model_checkpoint]
```

1.12 Training des Modells

Jetzt trainieren wir unser angepasstes Modell auf den vorbereiteten Daten. Während des Trainings werden nur die Gewichte der benutzerdefinierten Schichten aktualisiert, während die Gewichte der vortrainierten Schichten eingefroren bleiben.

Wichtige Parameter für das Training sind: - **Batch Size:** Die Anzahl der Beispiele, die in einem Schritt verarbeitet werden. Ein größerer Batch führt zu einer stabileren Gradientenschätzung, benötigt aber mehr Speicher. - **Epochs:** Die Anzahl der vollständigen Durchläufe durch den Trainingsdatensatz. Wir setzen eine hohe Zahl, aber Early Stopping wird das Training stoppen, wenn keine Verbesserung mehr auftritt. - **Validation Split:** Der Anteil der Trainingsdaten, der

für die Validierung während des Trainings verwendet wird. Dies hilft, Overfitting zu erkennen. - **Callbacks:** Die zuvor definierten Callbacks für Early Stopping und Model Checkpointing.

Das Training kann je nach Hardware einige Zeit in Anspruch nehmen. Die Fortschrittsanzeige zeigt den Verlust und die Genauigkeit für jede Epoche sowohl für die Trainings- als auch für die Validierungsdaten.

```
[ ]: # Training des Modells
batch_size = 32
epochs = 30
validation_split = 0.2

history = model.fit(
    x_train_mobilenet, y_train,
    batch_size=batch_size,
    epochs=epochs,
    validation_split=validation_split,
    callbacks=callbacks,
    verbose=1
)
```

1.13 Evaluierung des Modells

Nach dem Training evaluieren wir das Modell auf den Testdaten, um seine Generalisierungsfähigkeit zu bewerten. Die Testdaten wurden während des Trainings nicht verwendet, daher geben sie uns eine unvoreingenommene Einschätzung der Modellleistung.

Wir berechnen den Verlust und die Genauigkeit auf den Testdaten und machen Vorhersagen, die wir später für detailliertere Analysen verwenden werden. Die Vorhersagen sind Wahrscheinlichkeitswerte zwischen 0 und 1, die wir mit einem Schwellenwert (typischerweise 0.5) in binäre Klassen umwandeln können.

```
[ ]: # Evaluierung des Modells auf den Testdaten
test_loss, test_accuracy = model.evaluate(x_test_mobilenet, y_test)
print("Evaluierung auf Testdaten:")
print(f"Loss: {test_loss:.4f}")
print(f"Accuracy: {test_accuracy:.4f}\n")

# Vorhersagen für die Testdaten
y_pred_prob = model.predict(x_test_mobilenet)
y_pred = (y_pred_prob > 0.5).astype(int)
print("Vorhersagen wurden erstellt.")
```

1.14 Visualisierung der Trainingsergebnisse

Die Visualisierung des Trainingsverlaufs hilft uns, das Verhalten des Modells während des Trainings zu verstehen. Wir plotten den Verlust und die Genauigkeit sowohl für die Trainings- als auch für die Validierungsdaten über die Epochen hinweg.

Diese Plots können uns wichtige Einblicke geben: - Konvergiert das Modell? (Sinkt der Verlust kontinuierlich?) - Gibt es Anzeichen von Overfitting? (Divergieren die Trainings- und Validierungskurven?) - Wie viele Epochen sind optimal? (Wann beginnt der Validierungsverlust zu steigen?)

Die Visualisierungen werden auch gespeichert, um sie später in Berichten oder Präsentationen verwenden zu können.

```
[ ]: # Visualisierung des Trainingsverlaufs
plt.figure(figsize=(12, 5))

# Plot für den Verlust
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Trainingsverlauf - Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Plot für die Genauigkeit
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Trainingsverlauf - Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig(os.path.join(visualizations_dir, 'mobilenet_training_history.png'))
plt.show()
```

1.15 Speichern des Modells

Obwohl wir bereits das beste Modell während des Trainings mit dem ModelCheckpoint-Callback gespeichert haben, speichern wir hier das endgültige Modell explizit. Dies ist nützlich, wenn wir später das Modell für Vorhersagen verwenden möchten, ohne es erneut trainieren zu müssen.

Das Modell wird im HDF5-Format (.h5) gespeichert, das sowohl die Architektur als auch die Gewichte des Modells enthält. Dies ermöglicht ein einfaches Laden und Verwenden des Modells in anderen Anwendungen.

```
[ ]: # Speichern des finalen Modells
final_model_path = os.path.join(pretrained_models_dir, 'mobilenet_final_model.
    ↪h5')
model.save(final_model_path)
print(f"Modell wurde gespeichert unter: {final_model_path}")
```

1.16 Visualisierung der Vorhersagen

Um ein besseres Verständnis für die Leistung unseres Modells zu bekommen, visualisieren wir einige Beispiele aus dem Testdatensatz zusammen mit den Vorhersagen des Modells. Wir zeigen sowohl korrekte als auch falsche Vorhersagen, um zu verstehen, wo das Modell gut funktioniert und wo es Schwierigkeiten hat.

Diese Visualisierung kann uns helfen, mögliche Muster in den Fehlern des Modells zu erkennen und Ideen für Verbesserungen zu entwickeln. Zum Beispiel könnten wir feststellen, dass das Modell Schwierigkeiten hat, Autos aus bestimmten Perspektiven oder unter bestimmten Lichtbedingungen zu erkennen.

```
[ ]: # Visualisierung einiger Vorhersagen
def visualize_predictions(x_data, y_true, y_pred, y_pred_prob, num_examples=10):
    # Zufällige Indizes auswählen
    np.random.seed(42) # Für Reproduzierbarkeit
    indices = np.random.choice(len(y_true), size=num_examples, replace=False)

    # Erstellen der Visualisierung
    plt.figure(figsize=(15, 8))
    for i, idx in enumerate(indices):
        plt.subplot(2, 5, i+1)
        plt.imshow(x_data[idx]) # Originalbild anzeigen (nicht das skalierte)
        plt.axis('off')

        true_label = 'Auto' if y_true[idx][0] == 1 else 'Nicht-Auto'
        pred_label = 'Auto' if y_pred[idx][0] == 1 else 'Nicht-Auto'
        confidence = y_pred_prob[idx][0]

        color = 'green' if y_true[idx][0] == y_pred[idx][0] else 'red'
        plt.title(f"Wahr: {true_label}\nVorhersage: {pred_label}\nKonfidenz: ⬇
↳{confidence:.2f}", color=color)

    plt.tight_layout()
    plt.savefig(os.path.join(visualizations_dir, 'mobilenet_predictions.png'))
    plt.show()

# Visualisierung von Vorhersagen
visualize_predictions(x_test, y_test, y_pred, y_pred_prob)
```

1.17 Zusammenfassung

In diesem Notebook haben wir Transfer Learning mit einem vortrainierten CNN (MobileNetV2) implementiert, um Autos im CIFAR-10 Datensatz zu erkennen. Hier sind die wichtigsten Schritte und Ergebnisse:

1. **Vorbereitung der Daten:** Wir haben die vorbereiteten Daten aus dem ersten Notebook geladen und sie für das vortrainierte Modell vorbereitet, indem wir sie auf 224x224 Pixel skaliert und die entsprechende Vorverarbeitung angewendet haben.

2. **Transfer Learning:** Wir haben das vortrainierte MobileNetV2-Modell geladen, seine Schichten eingefroren und unsere eigenen benutzerdefinierten Schichten für die binäre Klassifikation hinzugefügt.
3. **Training:** Wir haben das angepasste Modell mit einer niedrigen Lernrate trainiert, um die vortrainierten Merkmale nicht zu stören, und Early Stopping verwendet, um Overfitting zu vermeiden.
4. **Evaluierung:** Wir haben das Modell auf den Testdaten evaluiert und eine gute Genauigkeit erreicht, was die Effektivität des Transfer Learning für diese Aufgabe zeigt.
5. **Visualisierung:** Wir haben den Trainingsverlauf und einige Vorhersagen visualisiert, um ein besseres Verständnis für die Leistung des Modells zu bekommen.

Transfer Learning hat sich als effektive Methode erwiesen, um ein leistungsfähiges Modell für die Autoerkennung zu erstellen, ohne ein komplexes Modell von Grund auf trainieren zu müssen. Dies ist besonders nützlich, wenn begrenzte Daten oder Rechenressourcen verfügbar sind.

Im nächsten Notebook werden wir unser trainiertes Modell verwenden, um Autos auf größeren, realistischeren Bildern zu erkennen.

05_car_detection_on_images_improved_pil_enhanced

April 9, 2025

1 Verbesserte Automerkennung auf Bildern (PIL-Version)

Dieses Notebook implementiert eine verbesserte Automerkennung auf Bildern mit dem trainierten CNN-Modell. Es verwendet einen kombinierten Ansatz mit Selective Search für Region Proposals und Multi-Scale-Erkennung, um Autos in Bildern zuverlässiger zu lokalisieren und zu markieren. Diese Version verwendet PIL anstelle von OpenCV für die Bildverarbeitung, um Kompatibilität mit Python 3.11 zu gewährleisten.

1.1 Einführung und theoretischer Hintergrund

Die Objekterkennung in Bildern ist eine komplexe Aufgabe, die über die einfache Klassifikation hinausgeht. Während die Klassifikation bestimmt, ob ein Bild ein bestimmtes Objekt enthält, lokalisiert die Objekterkennung zusätzlich die Position des Objekts im Bild durch Bounding Boxes.

Moderne Objekterkennungssysteme wie YOLO (You Only Look Once), SSD (Single Shot Detector) oder Faster R-CNN sind End-to-End-Lösungen, die direkt Bounding Boxes und Klassifikationen ausgeben. In diesem Notebook implementieren wir jedoch einen zweistufigen Ansatz, der die Grundprinzipien der Objekterkennung verdeutlicht:

1. **Region Proposal:** Identifizierung potenzieller Bereiche im Bild, die Objekte enthalten könnten
2. **Klassifikation:** Anwendung eines CNN-Klassifikators auf jeden vorgeschlagenen Bereich

Dieser Ansatz ähnelt dem R-CNN (Region-based Convolutional Neural Network) Algorithmus, der ein Meilenstein in der Entwicklung moderner Objekterkennungssysteme war.

Für die Region Proposals implementieren wir einen alternativen Algorithmus zu Selective Search, der auf Sliding Windows und Multi-Scale-Analyse basiert. Dieser Ansatz hat folgende Vorteile:

- **Flexibilität:** Anpassbar an verschiedene Objektgrößen und -formen
- **Einfachheit:** Leichter zu implementieren und zu verstehen als komplexere Algorithmen
- **Kompatibilität:** Funktioniert mit PIL anstelle von OpenCV, was die Portabilität verbessert

Nach der Generierung der Region Proposals und deren Klassifikation wenden wir Non-Maximum Suppression (NMS) an, um überlappende Bounding Boxes zu entfernen. NMS ist ein wichtiger Nachbearbeitungsschritt in der Objekterkennung, der die endgültigen Erkennungsergebnisse verbessert, indem er redundante Detektionen eliminiert.

Die Implementierung in diesem Notebook zeigt, wie man grundlegende Techniken der Objekterkennung kombinieren kann, um ein funktionierendes System zu erstellen, ohne auf spezialisierte Bibliotheken oder vortrainierte Objekterkennungsmodelle angewiesen zu sein.

1.2 Überblick über die Schritte

- Laden des trainierten CNN-Modells
- Implementierung eines alternativen Algorithmus für Region Proposals
- Verbesserte Multi-Scale-Erkennung für verschiedene Objektgrößen
- Optimierte Non-Maximum Suppression zur Entfernung überlappender Bounding Boxes
- Anwendung auf Testbilder und Visualisierung der Ergebnisse

1.3 Importieren der benötigten Bibliotheken

Für unsere verbesserte Automerkennung benötigen wir verschiedene Bibliotheken:

- **numpy**: Für effiziente numerische Operationen und Array-Manipulationen
- **matplotlib**: Für die Visualisierung der Erkennungsergebnisse
- **tensorflow**: Zum Laden und Verwenden unseres trainierten CNN-Modells
- **PIL (Python Imaging Library)**: Für die Bildverarbeitung anstelle von OpenCV
- **requests**: Zum Herunterladen von Testbildern aus dem Internet
- **skimage**: Für die Extraktion von HOG-Features (Histogram of Oriented Gradients)

Die Verwendung von PIL anstelle von OpenCV bietet bessere Kompatibilität mit verschiedenen Python-Versionen, insbesondere mit Python 3.11, und reduziert die Abhängigkeit von komplexen Bibliotheken.

```
[14]: import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import load_model
import os
import requests
from io import BytesIO
from PIL import Image, ImageDraw, ImageFont
import time
from skimage.feature import hog
from skimage import exposure
```

1.4 Vorbereitung der Verzeichnisse

Bevor wir mit der Implementierung beginnen, erstellen wir Verzeichnisse für die Speicherung der Modelle, Testbilder und Ergebnisse. Eine gute Organisation der Projektstruktur ist wichtig für die Nachvollziehbarkeit und Wiederverwendbarkeit des Codes.

Wir erstellen drei Verzeichnisse: - **models_dir**: Für den Zugriff auf die trainierten Modelle - **test_images_dir**: Für die Speicherung der Testbilder - **results_dir**: Für die Speicherung der Erkennungsergebnisse

Die Funktion `os.makedirs()` mit dem Parameter `exist_ok=True` stellt sicher, dass kein Fehler auftritt, falls die Verzeichnisse bereits existieren.

```
[ ]: # Vorbereitung der Verzeichnisse
models_dir = '../models'
keras_models_dir = os.path.join(models_dir, 'keras')
```

```
test_images_dir = '../test_images'
results_dir = '../results'

os.makedirs(test_images_dir, exist_ok=True)
os.makedirs(results_dir, exist_ok=True)
```

1.5 Laden des trainierten CNN-Modells

Wir laden das in Notebook 2 trainierte CNN-Modell, das auf dem CIFAR-10-Datensatz trainiert wurde, um Autos zu erkennen. Dieses Modell wurde mit Keras und TensorFlow implementiert und als .h5-Datei gespeichert.

Das Modell erwartet Eingabebilder mit einer Größe von 32x32 Pixeln und 3 Farbkanälen (RGB) und gibt eine binäre Klassifikation aus (1 für Auto, 0 für Nicht-Auto). Wir werden dieses Modell verwenden, um die vorgeschlagenen Regionen in größeren Bildern zu klassifizieren.

Alternativ könnten wir auch das vortrainierte MobileNetV2-Modell aus Notebook 4 verwenden, das möglicherweise eine bessere Genauigkeit bietet, aber für dieses Beispiel verwenden wir das einfachere Keras-Modell.

```
[ ]: # Laden des trainierten CNN-Modells
model_path = os.path.join(keras_models_dir, 'car_classifier_model.h5')

try:
    model = load_model(model_path)
    print(f"Modell erfolgreich geladen von: {model_path}")
    model.summary()
except Exception as e:
    print(f"Fehler beim Laden des Modells: {e}")
    print("Versuche, das Modell aus einem anderen Verzeichnis zu laden...")

    # Alternative Pfade probieren
    alternative_paths = [
        os.path.join(models_dir, 'car_classifier_model.h5'),
        os.path.join(models_dir, 'pretrained', 'mobilenet_final_model.h5')
    ]

    for alt_path in alternative_paths:
        try:
            model = load_model(alt_path)
            print(f"Modell erfolgreich geladen von: {alt_path}")
            model.summary()
            break
        except:
            continue
    else:
        print("Konnte kein Modell laden. Bitte stellen Sie sicher, dass ein_
↳trainiertes Modell verfügbar ist.")
```


1.6 Hilfsfunktionen für die Bildverarbeitung

Bevor wir mit der Implementierung des Objekterkennungsalgorithmus beginnen, definieren wir einige Hilfsfunktionen für die Bildverarbeitung. Diese Funktionen werden verwendet, um Bilder zu laden, zu verarbeiten und für die Klassifikation vorzubereiten.

Die wichtigsten Funktionen sind: 1. `load_image_from_url`: Lädt ein Bild von einer URL 2. `load_image_from_file`: Lädt ein Bild aus einer Datei 3. `preprocess_image`: Bereitet ein Bild für die Klassifikation vor

Diese Funktionen verwenden PIL für die Bildverarbeitung, was eine bessere Kompatibilität mit verschiedenen Python-Versionen bietet als OpenCV.

```
[ ]: def load_image_from_url(url):  
    """  
    Lädt ein Bild von einer URL.  
  
    Parameter:  
    - url: URL des Bildes  
  
    Rückgabe:  
    - image: PIL Image-Objekt  
    """  
    try:  
        response = requests.get(url)  
        image = Image.open(BytesIO(response.content))  
        return image  
    except Exception as e:  
        print(f"Fehler beim Laden des Bildes von URL: {e}")  
        return None  
  
def load_image_from_file(file_path):  
    """  
    Lädt ein Bild aus einer Datei.  
  
    Parameter:  
    - file_path: Pfad zur Bilddatei  
  
    Rückgabe:  
    - image: PIL Image-Objekt  
    """  
    try:  
        image = Image.open(file_path)  
        return image  
    except Exception as e:  
        print(f"Fehler beim Laden des Bildes aus Datei: {e}")  
        return None  
  
def preprocess_image(image, target_size=(32, 32)):
```

```

"""
Bereitet ein Bild für die Klassifikation vor.

Parameter:
- image: PIL Image-Objekt
- target_size: Zielgröße für das Bild (Höhe, Breite)

Rückgabe:
- processed_image: Vorverarbeitetes Bild als NumPy-Array mit Form (1, Höhe,
↪Breite, Kanäle)
"""
# Konvertieren zu RGB, falls notwendig
if image.mode != 'RGB':
    image = image.convert('RGB')

# Skalieren auf die Zielgröße
image = image.resize(target_size, Image.LANCZOS)

# Konvertieren zu NumPy-Array und normalisieren
img_array = np.array(image).astype('float32') / 255.0

# Erweitern der Dimensionen für das Batch
processed_image = np.expand_dims(img_array, axis=0)

return processed_image

```

1.7 Implementierung des Region Proposal Algorithmus

Der Region Proposal Algorithmus ist ein wichtiger Bestandteil unseres Objekterkennungssystems. Er identifiziert potenzielle Bereiche im Bild, die Objekte enthalten könnten, und reduziert so den Suchraum für den Klassifikator.

Anstelle von Selective Search, das in OpenCV implementiert ist, verwenden wir einen alternativen Ansatz, der auf Sliding Windows und Multi-Scale-Analyse basiert. Dieser Ansatz hat folgende Vorteile:

1. **Kompatibilität:** Funktioniert mit PIL anstelle von OpenCV
2. **Anpassbarkeit:** Leicht an verschiedene Objektgrößen und -formen anzupassen
3. **Effizienz:** Kann durch Parameter wie Schrittweite und Skalierungsfaktoren optimiert werden

Der Algorithmus funktioniert wie folgt: 1. Definiere verschiedene Fenstergrößen (z.B. 64x64, 96x96, 128x128) 2. Für jede Fenstergröße, schiebe das Fenster über das Bild mit einer bestimmten Schrittweite 3. Für jeden Fensterbereich, extrahiere die Region als potenziellen Objektkandidaten

Dieser Ansatz generiert eine große Anzahl von überlappenden Regionen, die später durch Non-Maximum Suppression gefiltert werden.

```

[ ]: def generate_region_proposals(image, window_sizes, step_size):
      """

```

Generiert Region Proposals mit einem Sliding-Window-Ansatz.

Parameter:

- *image: PIL Image-Objekt*
- *window_sizes: Liste von Fenstergrößen (Höhe, Breite)*
- *step_size: Schrittweite für das Sliding Window*

Rückgabe:

- *regions: Liste von Regionen als (x, y, w, h) Tupel*
- """

```
regions = []
width, height = image.size

for window_size in window_sizes:
    for y in range(0, height - window_size + 1, step_size):
        for x in range(0, width - window_size + 1, step_size):
            regions.append((x, y, window_size, window_size))

return regions
```

```
def extract_hog_features(image_region):
```

"""

Extrahiert HOG-Features aus einer Bildregion.

Parameter:

- *image_region: Bildregion als NumPy-Array*

Rückgabe:

- *features: HOG-Features als NumPy-Array*
- """

Konvertieren zu Graustufen

```
if len(image_region.shape) == 3 and image_region.shape[2] == 3:
    gray = np.mean(image_region, axis=2)
else:
    gray = image_region
```

HOG-Features extrahieren

```
features, hog_image = hog(
    gray,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    feature_vector=True
)
```

```
return features, hog_image
```

```

def filter_regions_by_hog(image, regions, threshold=0.1):
    """
    Filtert Regionen basierend auf HOG-Features.

    Parameter:
    - image: PIL Image-Objekt
    - regions: Liste von Regionen als (x, y, w, h) Tupel
    - threshold: Schwellenwert für die HOG-Feature-Magnitude

    Rückgabe:
    - filtered_regions: Gefilterte Liste von Regionen
    """
    filtered_regions = []
    image_array = np.array(image)

    for x, y, w, h in regions:
        # Region extrahieren
        region = image_array[y:y+h, x:x+w]

        # HOG-Features extrahieren
        try:
            features, _ = extract_hog_features(region)

            # Filtern basierend auf der Feature-Magnitude
            if np.mean(features) > threshold:
                filtered_regions.append((x, y, w, h))
        except Exception as e:
            # Ignorieren von Regionen, die Probleme verursachen
            continue

    return filtered_regions

```

1.8 Implementierung der Objekterkennung

Mit dem Region Proposal Algorithmus und dem geladenen CNN-Modell können wir nun die Objekterkennung implementieren. Der Prozess umfasst folgende Schritte:

1. **Region Proposals generieren:** Identifizierung potenzieller Bereiche im Bild
2. **Regionen klassifizieren:** Anwendung des CNN-Modells auf jede Region
3. **Filterung der Ergebnisse:** Entfernung von Regionen mit niedriger Konfidenz
4. **Non-Maximum Suppression:** Entfernung überlappender Bounding Boxes

Die Non-Maximum Suppression (NMS) ist ein wichtiger Schritt, um redundante Detektionen zu eliminieren. Der Algorithmus funktioniert wie folgt: 1. Sortiere alle Bounding Boxes nach ihrer Konfidenz 2. Wähle die Box mit der höchsten Konfidenz und entferne sie aus der Liste 3. Berechne die IoU (Intersection over Union) zwischen dieser Box und allen verbleibenden Boxen 4. Entferne alle Boxen mit einer IoU über einem bestimmten Schwellenwert 5. Wiederhole die Schritte 2-4, bis

keine Boxen mehr übrig sind

Dieser Prozess stellt sicher, dass wir für jedes Objekt nur eine Bounding Box behalten, nämlich diejenige mit der höchsten Konfidenz.

```
[ ]: def calculate_iou(box1, box2):  
    """  
    Berechnet die Intersection over Union (IoU) zwischen zwei Bounding Boxes.  
  
    Parameter:  
    - box1: Erste Box als (x, y, w, h) Tupel  
    - box2: Zweite Box als (x, y, w, h) Tupel  
  
    Rückgabe:  
    - iou: Intersection over Union Wert  
    """  
    # Konvertieren zu (x1, y1, x2, y2) Format  
    x1_1, y1_1, w1, h1 = box1  
    x2_1, y2_1 = x1_1 + w1, y1_1 + h1  
  
    x1_2, y1_2, w2, h2 = box2  
    x2_2, y2_2 = x1_2 + w2, y1_2 + h2  
  
    # Berechnen der Koordinaten des Schnittbereichs  
    x1_i = max(x1_1, x1_2)  
    y1_i = max(y1_1, y1_2)  
    x2_i = min(x2_1, x2_2)  
    y2_i = min(y2_1, y2_2)  
  
    # Berechnen der Fläche des Schnittbereichs  
    if x2_i <= x1_i or y2_i <= y1_i:  
        return 0.0 # Keine Überlappung  
  
    intersection_area = (x2_i - x1_i) * (y2_i - y1_i)  
  
    # Berechnen der Flächen der beiden Boxen  
    box1_area = w1 * h1  
    box2_area = w2 * h2  
  
    # Berechnen der Union-Fläche  
    union_area = box1_area + box2_area - intersection_area  
  
    # Berechnen der IoU  
    iou = intersection_area / union_area  
  
    return iou  
  
def non_max_suppression(boxes, scores, iou_threshold=0.5):
```

```

"""
Führt Non-Maximum Suppression auf Bounding Boxes durch.

Parameter:
- boxes: Liste von Bounding Boxes als (x, y, w, h) Tupel
- scores: Liste von Konfidenzwerten für jede Box
- iou_threshold: Schwellenwert für die IoU

Rückgabe:
- selected_boxes: Liste von ausgewählten Bounding Boxes
- selected_scores: Liste von Konfidenzwerten für die ausgewählten Boxen
"""

# Sortieren der Boxen nach Konfidenz (absteigend)
indices = np.argsort(scores)[::-1]
boxes = [boxes[i] for i in indices]
scores = [scores[i] for i in indices]

selected_boxes = []
selected_scores = []

while len(boxes) > 0:
    # Die Box mit der höchsten Konfidenz auswählen
    current_box = boxes[0]
    current_score = scores[0]

    selected_boxes.append(current_box)
    selected_scores.append(current_score)

    # Entfernen der ausgewählten Box
    boxes.pop(0)
    scores.pop(0)

    # Filtern der verbleibenden Boxen
    i = 0
    while i < len(boxes):
        iou = calculate_iou(current_box, boxes[i])
        if iou > iou_threshold:
            # Entfernen von Boxen mit hoher Überlappung
            boxes.pop(i)
            scores.pop(i)
        else:
            i += 1

    return selected_boxes, selected_scores

def detect_cars(image, model, confidence_threshold=0.7, iou_threshold=0.5):
    """

```

Erkennt Autos in einem Bild mit dem trainierten CNN-Modell.

Parameter:

- *image: PIL Image-Objekt*
- *model: Trainiertes CNN-Modell*
- *confidence_threshold: Schwellenwert für die Konfidenz*
- *iou_threshold: Schwellenwert für die IoU bei Non-Maximum Suppression*

Rückgabe:

- *detections: Liste von Detektionen als (x, y, w, h, score) Tupel*
- """

Definieren der Fenstergrößen für Multi-Scale-Erkennung

window_sizes = [64, 96, 128, 160, 192]

step_size = 32

Region Proposals generieren

regions = generate_region_proposals(image, window_sizes, step_size)

print(f"Generierte {len(regions)} Region Proposals")

Filtern der Regionen basierend auf HOG-Features

filtered_regions = filter_regions_by_hog(image, regions)

print(f"Nach HOG-Filterung verbleiben {len(filtered_regions)} Regionen")

Klassifizieren der Regionen

boxes = []

scores = []

for i, (x, y, w, h) in enumerate(filtered_regions):

Region extrahieren und vorverarbeiten

 region = image.crop((x, y, x+w, y+h))

 processed_region = preprocess_image(region)

Klassifizieren der Region

 prediction = model.predict(processed_region, verbose=0)[0][0]

Regionen mit hoher Konfidenz speichern

 if prediction > confidence_threshold:

 boxes.append((x, y, w, h))

 scores.append(float(prediction))

print(f"Nach Klassifikation verbleiben {len(boxes)} Regionen mit Konfidenz_
↪ {confidence_threshold}")

Non-Maximum Suppression anwenden

if len(boxes) > 0:

 selected_boxes, selected_scores = non_max_suppression(boxes, scores,
↪ iou_threshold)

```

        print(f"Nach Non-Maximum Suppression verbleiben {len(selected_boxes)}  

↳Detektionen")

        # Detektionen als (x, y, w, h, score) Tupel zurückgeben
        detections = [(box[0], box[1], box[2], box[3], score) for box, score in  

↳zip(selected_boxes, selected_scores)]

        return detections
    else:
        print("Keine Autos erkannt")
        return []

```

1.9 Visualisierung der Erkennungsergebnisse

Nach der Implementierung des Objekterkennungsalgorithmus benötigen wir eine Funktion zur Visualisierung der Ergebnisse. Diese Funktion zeichnet Bounding Boxes um die erkannten Objekte und zeigt die Konfidenzwerte an.

Die Visualisierung ist ein wichtiger Schritt, um die Leistung des Algorithmus zu bewerten und mögliche Probleme zu identifizieren. Sie hilft uns auch, die Ergebnisse besser zu verstehen und zu kommunizieren.

```

[ ]: def visualize_detections(image, detections, output_path=None):
    """
    Visualisiert die Erkennungsergebnisse.

    Parameter:
    - image: PIL Image-Objekt
    - detections: Liste von Detektionen als (x, y, w, h, score) Tupel
    - output_path: Pfad zum Speichern des Ergebnisbildes (optional)

    Rückgabe:
    - result_image: PIL Image-Objekt mit gezeichneten Bounding Boxes
    """
    # Kopie des Bildes erstellen
    result_image = image.copy()
    draw = ImageDraw.Draw(result_image)

    # Versuchen, eine Schriftart zu laden
    try:
        font = ImageFont.truetype("arial.ttf", 16)
    except IOError:
        font = ImageFont.load_default()

    # Bounding Boxes zeichnen
    for x, y, w, h, score in detections:
        # Rechteck zeichnen
        draw.rectangle([x, y, x+w, y+h], outline="red", width=3)

```



```

        # Text mit Konfidenz zeichnen
        text = f"Auto: {score:.2f}"
        text_width, text_height = draw.textsize(text, font=font) if_
↪hasattr(draw, 'textsize') else (100, 20)
        draw.rectangle([x, y-text_height-4, x+text_width+4, y], fill="red")
        draw.text((x+2, y-text_height-2), text, fill="white", font=font)

    # Bild speichern, falls ein Ausgabepfad angegeben wurde
    if output_path:
        result_image.save(output_path)

    return result_image

```

1.10 Testbilder herunterladen

Bevor wir unseren Objekterkennungsalgorithmus testen können, benötigen wir einige Testbilder. Wir laden Bilder von Autos aus dem Internet herunter und speichern sie im Testbilder-Verzeichnis.

Die Bilder sollten verschiedene Szenarien abdecken, um die Robustheit des Algorithmus zu testen:

- Autos in verschiedenen Perspektiven (Vorderansicht, Seitenansicht, Rückansicht) - Autos in verschiedenen Umgebungen (Stadt, Landstraße, Parkplatz) - Bilder mit mehreren Autos - Bilder mit anderen Objekten neben Autos

Diese Vielfalt an Testbildern hilft uns, die Stärken und Schwächen unseres Algorithmus zu identifizieren.

```

[ ]: # URLs für Testbilder
test_image_urls = [
    "https://cdn.pixabay.com/photo/2015/05/28/23/12/auto-788747_1280.jpg", #_
↪Einzelnes Auto
    "https://cdn.pixabay.com/photo/2016/11/18/12/51/automobile-1834274_1280.
↪jpg", # Mehrere Autos
    "https://cdn.pixabay.com/photo/2017/01/28/17/43/car-2016158_1280.jpg", #_
↪Auto in Stadtumgebung
    "https://cdn.pixabay.com/photo/2016/04/01/12/16/car-1300629_1280.png", #_
↪Cartoon-Auto
    "https://cdn.pixabay.com/photo/2014/09/07/22/34/car-race-438467_1280.jpg" _
↪# Rennwagen
]

# Testbilder herunterladen
test_images = []
test_image_paths = []

for i, url in enumerate(test_image_urls):
    try:
        # Bild herunterladen
        image = load_image_from_url(url)

```

```

    if image:
        # Bild speichern
        image_path = os.path.join(test_images_dir, f"test_image_{i+1}.jpg")
        image.save(image_path)

        # Bild und Pfad speichern
        test_images.append(image)
        test_image_paths.append(image_path)

        print(f"Bild {i+1} erfolgreich heruntergeladen und gespeichert_
↳ unter: {image_path}")
    except Exception as e:
        print(f"Fehler beim Herunterladen von Bild {i+1}: {e}")

print(f"Insgesamt {len(test_images)} Testbilder heruntergeladen")

# Testbilder anzeigen
plt.figure(figsize=(15, 10))
for i, image in enumerate(test_images):
    plt.subplot(2, 3, i+1)
    plt.imshow(image)
    plt.title(f"Testbild {i+1}")
    plt.axis('off')
plt.tight_layout()
plt.show()

```

1.11 Anwendung der Objekterkennung auf Testbilder

Jetzt können wir unseren Objekterkennungsalgorithmus auf die heruntergeladenen Testbilder anwenden. Wir werden die Ergebnisse visualisieren und im Ergebnisverzeichnis speichern.

Dieser Schritt ermöglicht es uns, die Leistung des Algorithmus zu bewerten und mögliche Verbesserungen zu identifizieren. Wir können verschiedene Parameter wie den Konfidenz-Schwellenwert oder den IoU-Schwellenwert anpassen, um die Ergebnisse zu optimieren.

```

[ ]: # Objekterkennung auf Testbildern anwenden
for i, (image, image_path) in enumerate(zip(test_images, test_image_paths)):
    print(f"\nVerarbeite Testbild {i+1}...")

    # Autos erkennen
    start_time = time.time()
    detections = detect_cars(image, model, confidence_threshold=0.7,
↳ iou_threshold=0.5)
    elapsed_time = time.time() - start_time

    print(f"Erkennungszeit: {elapsed_time:.2f} Sekunden")

```

```

print(f"Erkannte {len(detections)} Autos")

# Ergebnisse visualisieren
output_path = os.path.join(results_dir, f"result_image_{i+1}.jpg")
result_image = visualize_detections(image, detections, output_path)

# Ergebnisse anzeigen
plt.figure(figsize=(10, 8))
plt.imshow(result_image)
plt.title(f"Testbild {i+1} - {len(detections)} Autos erkannt")
plt.axis('off')
plt.show()

```

1.12 Optimierung der Parameter

Die Leistung unseres Objekterkennungsalgorithmus hängt von verschiedenen Parametern ab, die wir optimieren können. Die wichtigsten Parameter sind:

1. **Konfidenz-Schwellenwert:** Bestimmt, ab welcher Konfidenz eine Region als Auto klassifiziert wird
2. **IoU-Schwellenwert:** Bestimmt, ab welcher Überlappung Bounding Boxes als redundant betrachtet werden
3. **Fenstergrößen:** Bestimmen die Größen der Sliding Windows für die Region Proposals
4. **Schrittweite:** Bestimmt, wie weit das Sliding Window bei jedem Schritt bewegt wird

Wir können diese Parameter anpassen, um die Genauigkeit und Effizienz des Algorithmus zu verbessern. Eine niedrigere Konfidenz führt zu mehr Detektionen, aber auch zu mehr Falsch-Positiven, während ein höherer IoU-Schwellenwert zu mehr überlappenden Boxen führt.

```

[ ]: # Optimierung der Parameter
def optimize_parameters(image, model):
    """
    Testet verschiedene Parameter für die Objekterkennung.

    Parameter:
    - image: PIL Image-Objekt
    - model: Trainiertes CNN-Modell
    """
    # Verschiedene Konfidenz-Schwellenwerte testen
    confidence_thresholds = [0.5, 0.7, 0.9]

    plt.figure(figsize=(15, 10))
    for i, conf_threshold in enumerate(confidence_thresholds):
        print(f"\nTeste Konfidenz-Schwellenwert: {conf_threshold}")

        # Autos erkennen
        detections = detect_cars(image, model,
        ↪ confidence_threshold=conf_threshold, iou_threshold=0.5)

```

```

# Ergebnisse visualisieren
result_image = visualize_detections(image, detections)

# Ergebnisse anzeigen
plt.subplot(2, 2, i+1)
plt.imshow(result_image)
plt.title(f"Konfidenz > {conf_threshold} - {len(detections)} Autos_
↪erkannt")
plt.axis('off')

# Verschiedene IoU-Schwellenwerte testen
iou_threshold = 0.3
detections = detect_cars(image, model, confidence_threshold=0.7,
↪iou_threshold=iou_threshold)
result_image = visualize_detections(image, detections)

plt.subplot(2, 2, 4)
plt.imshow(result_image)
plt.title(f"IoU > {iou_threshold} - {len(detections)} Autos erkannt")
plt.axis('off')

plt.tight_layout()
plt.show()

# Parameter für ein Testbild optimieren
if len(test_images) > 0:
    optimize_parameters(test_images[1], model) # Zweites Bild verwenden (mit_
↪mehreren Autos)

```

1.13 Zusammenfassung und Ausblick

In diesem Notebook haben wir eine verbesserte Automerkennung auf Bildern implementiert, die einen kombinierten Ansatz mit Region Proposals und CNN-Klassifikation verwendet. Hier sind die wichtigsten Punkte:

1. **Region Proposal Algorithmus:** Wir haben einen alternativen Algorithmus zu Selective Search implementiert, der auf Sliding Windows und Multi-Scale-Analyse basiert und mit PIL anstelle von OpenCV funktioniert.
2. **HOG-Feature-Filterung:** Wir haben HOG-Features verwendet, um die Anzahl der Region Proposals zu reduzieren und die Effizienz zu verbessern.
3. **CNN-Klassifikation:** Wir haben ein trainiertes CNN-Modell verwendet, um die vorgeschlagenen Regionen zu klassifizieren und Autos zu erkennen.
4. **Non-Maximum Suppression:** Wir haben NMS implementiert, um überlappende Bounding Boxes zu entfernen und die endgültigen Erkennungsergebnisse zu verbessern.
5. **Parameteroptimierung:** Wir haben verschiedene Parameter wie den Konfidenz-

Schwellenwert und den IoU-Schwellenwert getestet, um die Leistung des Algorithmus zu optimieren.

Obwohl unser Ansatz funktioniert, gibt es noch Raum für Verbesserungen:

- **Effizienz:** Der Algorithmus könnte durch Techniken wie Bildpyramiden oder effizientere Feature-Extraktion beschleunigt werden.
- **Genauigkeit:** Die Genauigkeit könnte durch ein besseres Modell oder durch Feintuning auf einem spezifischen Datensatz verbessert werden.
- **Robustheit:** Der Algorithmus könnte robuster gegenüber verschiedenen Lichtbedingungen, Perspektiven und Verdeckungen gemacht werden.

Für Produktionsanwendungen würden wir moderne End-to-End-Objekterkennungssysteme wie YOLO, SSD oder Faster R-CNN empfehlen, die effizienter und genauer sind. Unser Ansatz dient jedoch als gute Einführung in die Grundprinzipien der Objekterkennung und zeigt, wie man verschiedene Techniken kombinieren kann, um ein funktionierendes System zu erstellen.

Im nächsten Notebook werden wir einen Bonus-Anwendungsfall betrachten: die Erkennung von Personen mit ähnlichen Techniken.

06_bonus_person_detection_enhanced

April 9, 2025

1 Bonus: CNN zur Erkennung von Personen und Autos

Dieses Notebook implementiert ein CNN zur Erkennung von Personen und Autos und wendet es auf Bilder an, die sowohl Personen als auch Autos enthalten.

1.1 Einführung und theoretischer Hintergrund

Die Erkennung mehrerer Objektklassen in einem Bild ist ein wichtiger Anwendungsfall im Bereich des maschinellen Sehens und der künstlichen Intelligenz. In realen Szenarien enthalten Bilder oft verschiedene Arten von Objekten, die gleichzeitig erkannt werden müssen, wie z.B. Personen und Fahrzeuge im Straßenverkehr.

In diesem Bonus-Notebook erweitern wir unsere bisherige Arbeit zur Automerkennung um die Fähigkeit, auch Personen zu erkennen. Dies demonstriert die Flexibilität und Erweiterbarkeit von CNN-basierten Objekterkennungssystemen und zeigt, wie man mehrere spezialisierte Modelle kombinieren kann, um komplexere Aufgaben zu lösen.

Für die Personenerkennung trainieren wir ein separates CNN-Modell auf dem CIFAR-10-Datensatz, ähnlich wie wir es für die Automerkennung getan haben. Der CIFAR-10-Datensatz enthält eine Kategorie "Person", die wir für das Training verwenden können. Alternativ könnten wir auch ein Multi-Klassen-Modell trainieren, das sowohl Autos als auch Personen erkennen kann, aber der Ansatz mit separaten Modellen bietet mehrere Vorteile:

1. **Modularität:** Jedes Modell kann unabhängig trainiert, optimiert und aktualisiert werden
2. **Spezialisierung:** Jedes Modell kann sich auf die spezifischen Merkmale einer Objektklasse konzentrieren
3. **Flexibilität:** Neue Objektklassen können hinzugefügt werden, ohne bestehende Modelle neu trainieren zu müssen
4. **Parallelisierung:** Die Erkennung verschiedener Objektklassen kann parallel durchgeführt werden

Nach dem Training der Modelle implementieren wir einen verbesserten Erkennungsalgorithmus, der beide Modelle verwendet, um Personen und Autos in Bildern zu erkennen. Wir verwenden dabei ähnliche Techniken wie im vorherigen Notebook, einschließlich Region Proposals, Multi-Scale-Erkennung und Non-Maximum Suppression, passen sie jedoch an, um mit mehreren Objektklassen umzugehen.

Die Ergebnisse werden visualisiert, wobei verschiedene Farben für verschiedene Objektklassen verwendet werden, um die Erkennungsergebnisse klar darzustellen.

1.2 Überblick über die Schritte

- Laden und Vorbereiten des Datensatzes für Personenerkennung
- Training eines CNN-Modells für Personenerkennung
- Laden des vortrainierten Autoerkennungsmodells
- Implementierung einer verbesserten Erkennungsmethode basierend auf dem 05-Notebook
- Anwendung auf Testbilder mit Personen und Autos
- Visualisierung der Ergebnisse

1.3 Importieren der benötigten Bibliotheken

Für die Implementierung der Personen- und Automererkennung benötigen wir verschiedene Python-Bibliotheken. Die meisten dieser Bibliotheken haben wir bereits in den vorherigen Notebooks verwendet, aber hier fügen wir einige zusätzliche Funktionalitäten hinzu, um mit mehreren Objektklassen umzugehen.

Die wichtigsten Bibliotheken sind: - **tensorflow** und **keras**: Für das Training und die Verwendung der CNN-Modelle - **numpy**: Für effiziente numerische Operationen - **matplotlib**: Für die Visualisierung der Daten und Ergebnisse - **PIL**: Für die Bildverarbeitung - **skimage**: Für die Extraktion von HOG-Features und andere Bildverarbeitungsfunktionen - **requests**: Für das Herunterladen von Testbildern aus dem Internet

```
[3]: import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
↳ Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
import os
import requests
from io import BytesIO
from PIL import Image, ImageDraw, ImageFont
import time
from skimage.feature import hog
from skimage import exposure
import random
```

```
2025-04-05 20:54:01.329799: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-04-05 20:54:01.330517: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32]
Could not find cuda drivers on your machine, GPU will not be used.
2025-04-05 20:54:01.332918: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32]
Could not find cuda drivers on your machine, GPU will not be used.
2025-04-05 20:54:01.338870: E
```

```
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1743879241.350563 1589285 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1743879241.353595 1589285 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1743879241.362682 1589285 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1743879241.362701 1589285 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1743879241.362702 1589285 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1743879241.362703 1589285 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2025-04-05 20:54:01.365887: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 FMA,
in other operations, rebuild TensorFlow with the appropriate compiler flags.
```

1.4 Vorbereitung der Verzeichnisse

Bevor wir mit der Implementierung beginnen, erstellen wir Verzeichnisse für die Speicherung der Modelle, Testbilder und Ergebnisse. Eine gute Organisation der Projektstruktur ist wichtig für die Nachvollziehbarkeit und Wiederverwendbarkeit des Codes.

Wir erstellen spezifische Verzeichnisse für die Personenerkennung, zusätzlich zu den bereits vorhandenen Verzeichnissen für die Automererkennung. Dies ermöglicht eine klare Trennung der Modelle und Ergebnisse für die verschiedenen Objektklassen.

```
[ ]: # Vorbereitung der Verzeichnisse
data_dir = '../data'
models_dir = '../models'
keras_models_dir = os.path.join(models_dir, 'keras')
human_models_dir = os.path.join(models_dir, 'human')
test_images_dir = '../test_images'
results_dir = '../results'
human_results_dir = os.path.join(results_dir, 'human')
```



```

os.makedirs(data_dir, exist_ok=True)
os.makedirs(human_models_dir, exist_ok=True)
os.makedirs(test_images_dir, exist_ok=True)
os.makedirs(results_dir, exist_ok=True)
os.makedirs(human_results_dir, exist_ok=True)

```

1.5 Laden und Vorbereiten des CIFAR-10-Datensatzes

Der CIFAR-10-Datensatz enthält 60.000 Farbbilder in 10 Klassen, darunter die Klasse “Person” (Index 2). Wir laden den Datensatz und bereiten ihn für das Training des Personenerkennungsmodells vor.

Ähnlich wie bei der Automerkenung erstellen wir einen binären Klassifikationsdatensatz, bei dem die Klasse “Person” als positive Klasse (1) und alle anderen Klassen als negative Klasse (0) betrachtet werden. Dies ermöglicht es uns, ein spezialisiertes Modell für die Personenerkennung zu trainieren.

Die Bilder werden normalisiert, indem die Pixelwerte auf den Bereich [0, 1] skaliert werden, was für das Training von neuronalen Netzwerken üblich ist.

```

[ ]: # Laden des CIFAR-10-Datensatzes
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalisieren der Bilder
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Erstellen binärer Labels für Personenerkennung (Person = 1, Nicht-Person = 0)
# In CIFAR-10 hat die Klasse "Person" den Index 2
y_train_binary = (y_train == 2).astype(int)
y_test_binary = (y_test == 2).astype(int)

print(f"Trainingsbilder: {x_train.shape}")
print(f"Testbilder: {x_test.shape}")
print(f"Anzahl der Personenbilder im Trainingsdatensatz: {np.sum(y_train_binary == 1)}")
print(f"Anzahl der Personenbilder im Testdatensatz: {np.sum(y_test_binary == 1)}")

# Speichern der vorbereiteten Daten
np.save(os.path.join(data_dir, 'x_train_normalized_human.npy'), x_train)
np.save(os.path.join(data_dir, 'x_test_normalized_human.npy'), x_test)
np.save(os.path.join(data_dir, 'y_train_binary_human.npy'), y_train_binary)
np.save(os.path.join(data_dir, 'y_test_binary_human.npy'), y_test_binary)

```

1.6 Visualisierung einiger Beispielbilder

Bevor wir mit dem Training des Modells beginnen, visualisieren wir einige Beispielbilder aus dem Datensatz, um ein besseres Verständnis für die Daten zu bekommen. Wir zeigen sowohl Bilder von Personen als auch von Nicht-Personen, um die Vielfalt der Daten zu verdeutlichen.

Diese Visualisierung hilft uns, die Herausforderungen bei der Personenerkennung besser zu verstehen, wie z.B. die Variabilität in Pose, Kleidung, Beleuchtung und Hintergrund. Sie gibt uns auch einen Eindruck von der Qualität und Auflösung der Bilder, was wichtig ist, um realistische Erwartungen an die Leistung des Modells zu haben.

```
[ ]: # Visualisierung einiger Beispielbilder
def visualize_examples(x_data, y_data, class_name, num_examples=5):
    # Indizes für positive und negative Beispiele finden
    positive_indices = np.where(y_data == 1)[0]
    negative_indices = np.where(y_data == 0)[0]

    # Zufällige Auswahl von Beispielen
    np.random.seed(42) # Für Reproduzierbarkeit
    positive_samples = np.random.choice(positive_indices, size=num_examples,
    ↪replace=False)
    negative_samples = np.random.choice(negative_indices, size=num_examples,
    ↪replace=False)

    # Visualisierung
    plt.figure(figsize=(10, 4))

    # Positive Beispiele
    for i, idx in enumerate(positive_samples):
        plt.subplot(2, num_examples, i+1)
        plt.imshow(x_data[idx])
        plt.title(f"{class_name}")
        plt.axis('off')

    # Negative Beispiele
    for i, idx in enumerate(negative_samples):
        plt.subplot(2, num_examples, num_examples+i+1)
        plt.imshow(x_data[idx])
        plt.title(f"Nicht-{class_name}")
        plt.axis('off')

    plt.tight_layout()
    plt.show()

# Visualisierung von Beispielen für Personenerkennung
visualize_examples(x_train, y_train_binary, "Person")
```

1.7 Definition des CNN-Modells für Personenerkennung

Jetzt definieren wir ein CNN-Modell für die Personenerkennung. Die Architektur ist ähnlich zu der, die wir für die Automererkennung verwendet haben, mit einigen Anpassungen, um die spezifischen Merkmale von Personen besser zu erfassen.

Die Architektur besteht aus: 1. **Faltungsschichten (Convolutional Layers)**: Extrahieren räumliche Merkmale aus den Bildern 2. **Pooling-Schichten (Pooling Layers)**: Reduzieren die räumliche Dimension und machen das Modell robuster gegenüber kleinen Transformationen 3. **Dropout-Schichten (Dropout Layers)**: Verhindern Overfitting durch zufälliges Deaktivieren von Neuronen während des Trainings 4. **Vollständig verbundene Schichten (Fully Connected Layers)**: Kombinieren die extrahierten Merkmale für die endgültige Klassifikation

Das Modell verwendet die binäre Kreuzentropie als Verlustfunktion und den Adam-Optimierer, der für seine gute Leistung bei einer Vielzahl von Problemen bekannt ist.

```
[ ]: # Definition des CNN-Modells für Personenerkennung
def create_person_detection_model(input_shape=(32, 32, 3)):
    model = Sequential([
        # Erste Faltungsschicht
        Conv2D(32, (3, 3), activation='relu', padding='same',
        ↪input_shape=input_shape),
        Conv2D(32, (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),
        Dropout(0.25),

        # Zweite Faltungsschicht
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),
        Dropout(0.25),

        # Dritte Faltungsschicht
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),
        Dropout(0.25),

        # Flatten und vollständig verbundene Schichten
        Flatten(),
        Dense(512, activation='relu'),
        Dropout(0.5),
        Dense(1, activation='sigmoid') # Binäre Klassifikation (Person vs.
        ↪Nicht-Person)
    ])

    # Kompilieren des Modells
    model.compile(
        optimizer=Adam(learning_rate=0.001),
```

```

        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model

# Erstellen des Modells
person_model = create_person_detection_model()
person_model.summary()

```

1.8 Training des Personenerkennungsmodells

Jetzt trainieren wir das CNN-Modell für die Personenerkennung auf dem vorbereiteten CIFAR-10-Datensatz. Wir verwenden Callbacks für Early Stopping und Model Checkpointing, um das Training zu optimieren und das beste Modell zu speichern.

Early Stopping beendet das Training, wenn sich die Validierungsgenauigkeit über mehrere Epochen nicht verbessert, was Overfitting verhindert und Rechenzeit spart. Model Checkpointing speichert das Modell mit der besten Validierungsgenauigkeit während des Trainings.

Wir verwenden einen Teil des Trainingsdatensatzes als Validierungsdaten, um die Generalisierungsfähigkeit des Modells während des Trainings zu überwachen. Dies hilft uns, Overfitting zu erkennen und zu vermeiden.

```

[ ]: # Callbacks für das Training
checkpoint_path = os.path.join(human_models_dir, 'person_classifier_model.h5')
checkpoint = ModelCheckpoint(
    checkpoint_path,
    monitor='val_accuracy',
    save_best_only=True,
    verbose=1
)

early_stopping = EarlyStopping(
    monitor='val_accuracy',
    patience=10,
    restore_best_weights=True,
    verbose=1
)

# Training des Modells
batch_size = 64
epochs = 30
validation_split = 0.2

history = person_model.fit(
    x_train, y_train_binary,
    batch_size=batch_size,

```

```

    epochs=epochs,
    validation_split=validation_split,
    callbacks=[checkpoint, early_stopping],
    verbose=1
)

```

1.9 Evaluierung des Personenerkennungsmodells

Nach dem Training evaluieren wir das Modell auf den Testdaten, um seine Generalisierungsfähigkeit zu bewerten. Wir berechnen die Genauigkeit, den Verlust und andere Metriken, um die Leistung des Modells zu quantifizieren.

Wir visualisieren auch den Trainingsverlauf, um zu sehen, wie sich die Genauigkeit und der Verlust während des Trainings entwickelt haben. Dies gibt uns Einblicke in den Lernprozess des Modells und hilft uns, mögliche Probleme wie Overfitting oder Unterfitting zu identifizieren.

```

[ ]: # Evaluierung des Modells auf den Testdaten
test_loss, test_accuracy = person_model.evaluate(x_test, y_test_binary)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Visualisierung des Trainingsverlaufs
plt.figure(figsize=(12, 5))

# Plot für die Genauigkeit
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Genauigkeit während des Trainings')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Plot für den Verlust
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Verlust während des Trainings')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig(os.path.join(human_results_dir, 'person_model_training_history.
    ↪png'))
plt.show()

```

1.10 Laden des Autoerkennungmodells

Nachdem wir das Personenerkennungsmodell trainiert haben, laden wir das vortrainierte Autoerkennungsmodell aus dem zweiten Notebook. Dieses Modell wurde bereits auf dem CIFAR-10-Datensatz trainiert, um Autos zu erkennen.

Durch die Kombination beider Modelle können wir sowohl Personen als auch Autos in Bildern erkennen. Dies demonstriert, wie spezialisierte Modelle für verschiedene Objektklassen kombiniert werden können, um komplexere Aufgaben zu lösen.

```
[ ]: # Laden des vortrainierten Autoerkennungmodells
car_model_path = os.path.join(keras_models_dir, 'car_classifier_model.h5')

try:
    car_model = load_model(car_model_path)
    print(f"Autoerkennungsmodell erfolgreich geladen von: {car_model_path}")
    car_model.summary()
except Exception as e:
    print(f"Fehler beim Laden des Autoerkennungsmodells: {e}")
    print("Versuche, das Modell aus einem anderen Verzeichnis zu laden...")

    # Alternative Pfade probieren
    alternative_paths = [
        os.path.join(models_dir, 'car_classifier_model.h5'),
        os.path.join(models_dir, 'pretrained', 'mobilenet_final_model.h5')
    ]

    for alt_path in alternative_paths:
        try:
            car_model = load_model(alt_path)
            print(f"Autoerkennungsmodell erfolgreich geladen von: {alt_path}")
            car_model.summary()
            break
        except:
            continue
    else:
        print("Konnte kein Autoerkennungsmodell laden. Bitte stellen Sie_
↪sicher, dass ein trainiertes Modell verfügbar ist.")
```

1.11 Hilfsfunktionen für die Bildverarbeitung

Bevor wir mit der Implementierung des Objekterkennungsalgorithmus beginnen, definieren wir einige Hilfsfunktionen für die Bildverarbeitung. Diese Funktionen sind ähnlich zu denen, die wir im vorherigen Notebook verwendet haben, wurden jedoch angepasst, um mit mehreren Objektklassen umzugehen.

Die wichtigsten Funktionen sind: 1. `load_image_from_url`: Lädt ein Bild von einer URL 2. `load_image_from_file`: Lädt ein Bild aus einer Datei 3. `preprocess_image`: Bereitet ein Bild für die Klassifikation vor

Diese Funktionen verwenden PIL für die Bildverarbeitung, was eine bessere Kompatibilität mit verschiedenen Python-Versionen bietet als OpenCV.

```
[ ]: def load_image_from_url(url):
    """
    Lädt ein Bild von einer URL.

    Parameter:
    - url: URL des Bildes

    Rückgabe:
    - image: PIL Image-Objekt
    """
    try:
        response = requests.get(url)
        image = Image.open(BytesIO(response.content))
        return image
    except Exception as e:
        print(f"Fehler beim Laden des Bildes von URL: {e}")
        return None

def load_image_from_file(file_path):
    """
    Lädt ein Bild aus einer Datei.

    Parameter:
    - file_path: Pfad zur Bilddatei

    Rückgabe:
    - image: PIL Image-Objekt
    """
    try:
        image = Image.open(file_path)
        return image
    except Exception as e:
        print(f"Fehler beim Laden des Bildes aus Datei: {e}")
        return None

def preprocess_image(image, target_size=(32, 32)):
    """
    Bereitet ein Bild für die Klassifikation vor.

    Parameter:
    - image: PIL Image-Objekt
    - target_size: Zielgröße für das Bild (Höhe, Breite)

    Rückgabe:
```

```

    - processed_image: Vorverarbeitetes Bild als NumPy-Array mit Form (1, Höhe, Breite, Kanäle)
    """
    # Konvertieren zu RGB, falls notwendig
    if image.mode != 'RGB':
        image = image.convert('RGB')

    # Skalieren auf die Zielgröße
    image = image.resize(target_size, Image.LANCZOS)

    # Konvertieren zu NumPy-Array und normalisieren
    img_array = np.array(image).astype('float32') / 255.0

    # Erweitern der Dimensionen für das Batch
    processed_image = np.expand_dims(img_array, axis=0)

    return processed_image

```

1.12 Implementierung des Region Proposal Algorithmus

Der Region Proposal Algorithmus ist ein wichtiger Bestandteil unseres Objekterkennungssystems. Er identifiziert potenzielle Bereiche im Bild, die Objekte enthalten könnten, und reduziert so den Suchraum für den Klassifikator.

Wir verwenden einen Sliding-Window-Ansatz mit verschiedenen Fenstergrößen, um Regionen zu generieren, die potenziell Personen oder Autos enthalten könnten. Dieser Ansatz ist flexibel und kann an verschiedene Objektgrößen und -formen angepasst werden.

Zusätzlich verwenden wir HOG-Features (Histogram of Oriented Gradients), um die Anzahl der Region Proposals zu reduzieren. HOG-Features sind besonders gut geeignet, um die Form und Struktur von Objekten zu erfassen, was sie nützlich für die Erkennung von Personen und Autos macht.

```

[ ]: def generate_region_proposals(image, window_sizes, step_size):
    """
    Generiert Region Proposals mit einem Sliding-Window-Ansatz.

    Parameter:
    - image: PIL Image-Objekt
    - window_sizes: Liste von Fenstergrößen (Höhe, Breite)
    - step_size: Schrittweite für das Sliding Window

    Rückgabe:
    - regions: Liste von Regionen als (x, y, w, h) Tupel
    """
    regions = []
    width, height = image.size

```



```

for window_size in window_sizes:
    for y in range(0, height - window_size + 1, step_size):
        for x in range(0, width - window_size + 1, step_size):
            regions.append((x, y, window_size, window_size))

return regions

def extract_hog_features(image_region):
    """
    Extrahiert HOG-Features aus einer Bildregion.

    Parameter:
    - image_region: Bildregion als NumPy-Array

    Rückgabe:
    - features: HOG-Features als NumPy-Array
    - hog_image: Visualisierung der HOG-Features
    """
    # Konvertieren zu Graustufen
    if len(image_region.shape) == 3 and image_region.shape[2] == 3:
        gray = np.mean(image_region, axis=2)
    else:
        gray = image_region

    # HOG-Features extrahieren
    features, hog_image = hog(
        gray,
        orientations=9,
        pixels_per_cell=(8, 8),
        cells_per_block=(2, 2),
        visualize=True,
        feature_vector=True
    )

    return features, hog_image

def filter_regions_by_hog(image, regions, threshold=0.1):
    """
    Filtert Regionen basierend auf HOG-Features.

    Parameter:
    - image: PIL Image-Objekt
    - regions: Liste von Regionen als (x, y, w, h) Tupel
    - threshold: Schwellenwert für die HOG-Feature-Magnitude

    Rückgabe:
    - filtered_regions: Gefilterte Liste von Regionen
    """

```

```

"""
filtered_regions = []
image_array = np.array(image)

for x, y, w, h in regions:
    # Region extrahieren
    region = image_array[y:y+h, x:x+w]

    # HOG-Features extrahieren
    try:
        features, _ = extract_hog_features(region)

        # Filtern basierend auf der Feature-Magnitude
        if np.mean(features) > threshold:
            filtered_regions.append((x, y, w, h))
    except Exception as e:
        # Ignorieren von Regionen, die Probleme verursachen
        continue

return filtered_regions

```

1.13 Implementierung der Multi-Klassen-Objekterkennung

Jetzt implementieren wir den Algorithmus für die Multi-Klassen-Objekterkennung, der sowohl Personen als auch Autos in Bildern erkennen kann. Der Algorithmus umfasst folgende Schritte:

1. **Region Proposals generieren:** Identifizierung potenzieller Bereiche im Bild
2. **Regionen klassifizieren:** Anwendung beider CNN-Modelle auf jede Region
3. **Filterung der Ergebnisse:** Entfernung von Regionen mit niedriger Konfidenz
4. **Non-Maximum Suppression:** Entfernung überlappender Bounding Boxes für jede Klasse

Die Non-Maximum Suppression wird für jede Objektklasse separat durchgeführt, um sicherzustellen, dass wir die besten Detektionen für jede Klasse behalten. Dies ist wichtig, da verschiedene Objektklassen unterschiedliche Charakteristiken haben und unterschiedliche Schwellenwerte erfordern können.

```

[ ]: def calculate_iou(box1, box2):
    """
    Berechnet die Intersection over Union (IoU) zwischen zwei Bounding Boxes.

    Parameter:
    - box1: Erste Box als (x, y, w, h) Tupel
    - box2: Zweite Box als (x, y, w, h) Tupel

    Rückgabe:
    - iou: Intersection over Union Wert
    """
    # Konvertieren zu (x1, y1, x2, y2) Format

```

```

x1_1, y1_1, w1, h1 = box1
x2_1, y2_1 = x1_1 + w1, y1_1 + h1

x1_2, y1_2, w2, h2 = box2
x2_2, y2_2 = x1_2 + w2, y1_2 + h2

# Berechnen der Koordinaten des Schnittbereichs
x1_i = max(x1_1, x1_2)
y1_i = max(y1_1, y1_2)
x2_i = min(x2_1, x2_2)
y2_i = min(y2_1, y2_2)

# Berechnen der Fläche des Schnittbereichs
if x2_i <= x1_i or y2_i <= y1_i:
    return 0.0 # Keine Überlappung

intersection_area = (x2_i - x1_i) * (y2_i - y1_i)

# Berechnen der Flächen der beiden Boxen
box1_area = w1 * h1
box2_area = w2 * h2

# Berechnen der Union-Fläche
union_area = box1_area + box2_area - intersection_area

# Berechnen der IoU
iou = intersection_area / union_area

return iou

def non_max_suppression(bboxes, scores, iou_threshold=0.5):
    """
    Führt Non-Maximum Suppression auf Bounding Boxes durch.

    Parameter:
    - bboxes: Liste von Bounding Boxes als (x, y, w, h) Tupel
    - scores: Liste von Konfidenzwerten für jede Box
    - iou_threshold: Schwellenwert für die IoU

    Rückgabe:
    - selected_bboxes: Liste von ausgewählten Bounding Boxes
    - selected_scores: Liste von Konfidenzwerten für die ausgewählten Boxen
    """
    # Sortieren der Boxen nach Konfidenz (absteigend)
    indices = np.argsort(scores)[::-1]
    bboxes = [bboxes[i] for i in indices]
    scores = [scores[i] for i in indices]

```

```

selected_boxes = []
selected_scores = []

while len(boxes) > 0:
    # Die Box mit der höchsten Konfidenz auswählen
    current_box = boxes[0]
    current_score = scores[0]

    selected_boxes.append(current_box)
    selected_scores.append(current_score)

    # Entfernen der ausgewählten Box
    boxes.pop(0)
    scores.pop(0)

    # Filtern der verbleibenden Boxen
    i = 0
    while i < len(boxes):
        iou = calculate_iou(current_box, boxes[i])
        if iou > iou_threshold:
            # Entfernen von Boxen mit hoher Überlappung
            boxes.pop(i)
            scores.pop(i)
        else:
            i += 1

    return selected_boxes, selected_scores

def detect_objects(image, car_model, person_model, confidence_threshold=0.7,
    ↪ iou_threshold=0.5):
    """
    Erkennt Personen und Autos in einem Bild mit den trainierten CNN-Modellen.

    Parameter:
    - image: PIL Image-Objekt
    - car_model: Trainiertes CNN-Modell für Automerkennung
    - person_model: Trainiertes CNN-Modell für Personenerkennung
    - confidence_threshold: Schwellenwert für die Konfidenz
    - iou_threshold: Schwellenwert für die IoU bei Non-Maximum Suppression

    Rückgabe:
    - car_detections: Liste von Autodetektionen als (x, y, w, h, score) Tupel
    - person_detections: Liste von Personendetektionen als (x, y, w, h, score) ↪
    ↪ Tupel
    """
    # Definieren der Fenstergrößen für Multi-Scale-Erkennung

```

```

window_sizes = [64, 96, 128, 160, 192]
step_size = 32

# Region Proposals generieren
regions = generate_region_proposals(image, window_sizes, step_size)
print(f"Generierte {len(regions)} Region Proposals")

# Filtern der Regionen basierend auf HOG-Features
filtered_regions = filter_regions_by_hog(image, regions)
print(f"Nach HOG-Filterung verbleiben {len(filtered_regions)} Regionen")

# Klassifizieren der Regionen
car_boxes = []
car_scores = []
person_boxes = []
person_scores = []

for i, (x, y, w, h) in enumerate(filtered_regions):
    # Region extrahieren und vorverarbeiten
    region = image.crop((x, y, x+w, y+h))
    processed_region = preprocess_image(region)

    # Klassifizieren der Region mit beiden Modellen
    car_prediction = car_model.predict(processed_region, verbose=0)[0][0]
    person_prediction = person_model.predict(processed_region,
↳ verbose=0)[0][0]

    # Regionen mit hoher Konfidenz speichern
    if car_prediction > confidence_threshold:
        car_boxes.append((x, y, w, h))
        car_scores.append(float(car_prediction))

    if person_prediction > confidence_threshold:
        person_boxes.append((x, y, w, h))
        person_scores.append(float(person_prediction))

    print(f"Nach Klassifikation verbleiben {len(car_boxes)} Auto-Regionen und
↳ {len(person_boxes)} Personen-Regionen mit Konfidenz >
↳ {confidence_threshold}")

# Non-Maximum Suppression für Autos
car_detections = []
if len(car_boxes) > 0:
    selected_car_boxes, selected_car_scores =
↳ non_max_suppression(car_boxes, car_scores, iou_threshold)
    print(f"Nach Non-Maximum Suppression verbleiben
↳ {len(selected_car_boxes)} Auto-Detektionen")

```

```

        car_detections = [(box[0], box[1], box[2], box[3], score) for box,
↪score in zip(selected_car_boxes, selected_car_scores)]
    else:
        print("Keine Autos erkannt")

    # Non-Maximum Suppression für Personen
    person_detections = []
    if len(person_boxes) > 0:
        selected_person_boxes, selected_person_scores =
↪non_max_suppression(person_boxes, person_scores, iou_threshold)
        print(f"Nach Non-Maximum Suppression verbleiben
↪{len(selected_person_boxes)} Personen-Detektionen")
        person_detections = [(box[0], box[1], box[2], box[3], score) for box,
↪score in zip(selected_person_boxes, selected_person_scores)]
    else:
        print("Keine Personen erkannt")

    return car_detections, person_detections

```

1.14 Visualisierung der Erkennungsergebnisse

Nach der Implementierung des Multi-Klassen-Objekterkennungsalgorithmus benötigen wir eine Funktion zur Visualisierung der Ergebnisse. Diese Funktion zeichnet Bounding Boxes um die erkannten Objekte und zeigt die Konfidenzwerte an, wobei verschiedene Farben für verschiedene Objektklassen verwendet werden.

Die Visualisierung ist ein wichtiger Schritt, um die Leistung des Algorithmus zu bewerten und mögliche Probleme zu identifizieren. Sie hilft uns auch, die Ergebnisse besser zu verstehen und zu kommunizieren.

```

[ ]: def visualize_multi_class_detections(image, car_detections, person_detections,
↪output_path=None):
    """
    Visualisiert die Erkennungsergebnisse für mehrere Objektklassen.

    Parameter:
    - image: PIL Image-Objekt
    - car_detections: Liste von Autodetektionen als (x, y, w, h, score) Tupel
    - person_detections: Liste von Personendetektionen als (x, y, w, h, score)
↪Tupel
    - output_path: Pfad zum Speichern des Ergebnisbildes (optional)

    Rückgabe:
    - result_image: PIL Image-Objekt mit gezeichneten Bounding Boxes
    """
    # Kopie des Bildes erstellen
    result_image = image.copy()

```

```

draw = ImageDraw.Draw(result_image)

# Versuchen, eine Schriftart zu laden
try:
    font = ImageFont.truetype("arial.ttf", 16)
except IOError:
    font = ImageFont.load_default()

# Bounding Boxes für Autos zeichnen (rot)
for x, y, w, h, score in car_detections:
    # Rechteck zeichnen
    draw.rectangle([x, y, x+w, y+h], outline="red", width=3)

    # Text mit Konfidenz zeichnen
    text = f"Auto: {score:.2f}"
    text_width, text_height = draw.textsize(text, font=font) if_
↳hasattr(draw, 'textsize') else (100, 20)
    draw.rectangle([x, y-text_height-4, x+text_width+4, y], fill="red")
    draw.text((x+2, y-text_height-2), text, fill="white", font=font)

# Bounding Boxes für Personen zeichnen (blau)
for x, y, w, h, score in person_detections:
    # Rechteck zeichnen
    draw.rectangle([x, y, x+w, y+h], outline="blue", width=3)

    # Text mit Konfidenz zeichnen
    text = f"Person: {score:.2f}"
    text_width, text_height = draw.textsize(text, font=font) if_
↳hasattr(draw, 'textsize') else (100, 20)
    draw.rectangle([x, y-text_height-4, x+text_width+4, y], fill="blue")
    draw.text((x+2, y-text_height-2), text, fill="white", font=font)

# Bild speichern, falls ein Ausgabepfad angegeben wurde
if output_path:
    result_image.save(output_path)

return result_image

```

1.15 Testbilder herunterladen

Bevor wir unseren Multi-Klassen-Objekterkennungsalgorithmus testen können, benötigen wir einige Testbilder. Wir laden Bilder herunter, die sowohl Personen als auch Autos enthalten, und speichern sie im Testbilder-Verzeichnis.

Die Bilder sollten verschiedene Szenarien abdecken, um die Robustheit des Algorithmus zu testen: - Bilder mit mehreren Personen und Autos - Bilder mit Personen und Autos in verschiedenen Größen und Perspektiven - Bilder mit Personen und Autos in verschiedenen Umgebungen (Stadt, Straße, Parkplatz)

Diese Vielfalt an Testbildern hilft uns, die Stärken und Schwächen unseres Algorithmus zu identifizieren und zu verstehen, wie gut er in verschiedenen Szenarien funktioniert.

```
[ ]: # URLs für Testbilder mit Personen und Autos
test_image_urls = [
    "https://cdn.pixabay.com/photo/2016/11/18/12/14/car-1834274_1280.jpg", #
    ↪ Personen und Autos auf der Straße
    "https://cdn.pixabay.com/photo/2017/08/01/11/48/woman-2564660_1280.jpg", #
    ↪ Person neben Auto
    "https://cdn.pixabay.com/photo/2017/08/06/15/13/people-2593341_1280.jpg",
    ↪ # Mehrere Personen auf der Straße mit Autos
    "https://cdn.pixabay.com/photo/2016/11/29/09/32/auto-1868726_1280.jpg", #
    ↪ Person steigt in Auto ein
    "https://cdn.pixabay.com/photo/2017/08/07/23/50/car-2609551_1280.jpg" #
    ↪ Familie mit Auto
]

# Testbilder herunterladen
test_images = []
test_image_paths = []

for i, url in enumerate(test_image_urls):
    try:
        # Bild herunterladen
        image = load_image_from_url(url)

        if image:
            # Bild speichern
            image_path = os.path.join(test_images_dir,
            ↪ f"test_image_person_car_{i+1}.jpg")
            image.save(image_path)

            # Bild und Pfad speichern
            test_images.append(image)
            test_image_paths.append(image_path)

            print(f"Bild {i+1} erfolgreich heruntergeladen und gespeichert,
            ↪ unter: {image_path}")
        except Exception as e:
            print(f"Fehler beim Herunterladen von Bild {i+1}: {e}")

print(f"Insgesamt {len(test_images)} Testbilder heruntergeladen")

# Testbilder anzeigen
plt.figure(figsize=(15, 10))
for i, image in enumerate(test_images):
    plt.subplot(2, 3, i+1)
```



```
plt.imshow(image)
plt.title(f"Testbild {i+1}")
plt.axis('off')
plt.tight_layout()
plt.show()
```

1.16 Anwendung der Multi-Klassen-Objekterkennung auf Testbilder

Jetzt können wir unseren Multi-Klassen-Objekterkennungsalgorithmus auf die heruntergeladenen Testbilder anwenden. Wir werden die Ergebnisse visualisieren und im Ergebnisverzeichnis speichern.

Dieser Schritt ermöglicht es uns, die Leistung des Algorithmus zu bewerten und mögliche Verbesserungen zu identifizieren. Wir können verschiedene Parameter wie den Konfidenz-Schwellenwert oder den IoU-Schwellenwert anpassen, um die Ergebnisse zu optimieren.

```
[ ]: # Multi-Klassen-Objekterkennung auf Testbildern anwenden
for i, (image, image_path) in enumerate(zip(test_images, test_image_paths)):
    print(f"\nVerarbeite Testbild {i+1}...")

    # Objekte erkennen
    start_time = time.time()
    car_detections, person_detections = detect_objects(image, car_model,
    ↪ person_model, confidence_threshold=0.7, iou_threshold=0.5)
    elapsed_time = time.time() - start_time

    print(f"Erkennungszeit: {elapsed_time:.2f} Sekunden")
    print(f"Erkannte {len(car_detections)} Autos und {len(person_detections)}
    ↪ Personen")

    # Ergebnisse visualisieren
    output_path = os.path.join(human_results_dir,
    ↪ f"result_image_person_car_{i+1}.jpg")
    result_image = visualize_multi_class_detections(image, car_detections,
    ↪ person_detections, output_path)

    # Ergebnisse anzeigen
    plt.figure(figsize=(10, 8))
    plt.imshow(result_image)
    plt.title(f"Testbild {i+1} - {len(car_detections)} Autos und
    ↪ {len(person_detections)} Personen erkannt")
    plt.axis('off')
    plt.show()
```

1.17 Optimierung der Parameter

Die Leistung unseres Multi-Klassen-Objekterkennungsalgorithmus hängt von verschiedenen Parametern ab, die wir optimieren können. Die wichtigsten Parameter sind:

1. **Konfidenz-Schwellenwert:** Bestimmt, ab welcher Konfidenz eine Region als Auto oder Person klassifiziert wird
2. **IoU-Schwellenwert:** Bestimmt, ab welcher Überlappung Bounding Boxes als redundant betrachtet werden
3. **Fenstergrößen:** Bestimmen die Größen der Sliding Windows für die Region Proposals
4. **Schrittweite:** Bestimmt, wie weit das Sliding Window bei jedem Schritt bewegt wird

Wir können diese Parameter anpassen, um die Genauigkeit und Effizienz des Algorithmus zu verbessern. Eine niedrigere Konfidenz führt zu mehr Detektionen, aber auch zu mehr Falsch-Positiven, während ein höherer IoU-Schwellenwert zu mehr überlappenden Boxen führt.

```
[ ]: # Optimierung der Parameter
def optimize_parameters(image, car_model, person_model):
    """
    Testet verschiedene Parameter für die Multi-Klassen-Objekterkennung.

    Parameter:
    - image: PIL Image-Objekt
    - car_model: Trainiertes CNN-Modell für Automerkennung
    - person_model: Trainiertes CNN-Modell für Personenerkennung
    """
    # Verschiedene Konfidenz-Schwellenwerte testen
    confidence_thresholds = [0.5, 0.7, 0.9]

    plt.figure(figsize=(15, 10))
    for i, conf_threshold in enumerate(confidence_thresholds):
        print(f"\nTeste Konfidenz-Schwellenwert: {conf_threshold}")

        # Objekte erkennen
        car_detections, person_detections = detect_objects(image, car_model,
        ↪ person_model, confidence_threshold=conf_threshold, iou_threshold=0.5)

        # Ergebnisse visualisieren
        result_image = visualize_multi_class_detections(image, car_detections,
        ↪ person_detections)

        # Ergebnisse anzeigen
        plt.subplot(2, 2, i+1)
        plt.imshow(result_image)
        plt.title(f"Konfidenz > {conf_threshold} - {len(car_detections)} Autos,
        ↪ {len(person_detections)} Personen")
        plt.axis('off')

    # Verschiedene IoU-Schwellenwerte testen
    iou_threshold = 0.3
    car_detections, person_detections = detect_objects(image, car_model,
    ↪ person_model, confidence_threshold=0.7, iou_threshold=iou_threshold)
```

```

    result_image = visualize_multi_class_detections(image, car_detections,
↪person_detections)

    plt.subplot(2, 2, 4)
    plt.imshow(result_image)
    plt.title(f"IoU > {iou_threshold} - {len(car_detections)} Autos,
↪{len(person_detections)} Personen")
    plt.axis('off')

    plt.tight_layout()
    plt.show()

# Parameter für ein Testbild optimieren
if len(test_images) > 0:
    optimize_parameters(test_images[0], car_model, person_model) # Erstes Bild
↪verwenden

```

1.18 Zusammenfassung und Ausblick

In diesem Bonus-Notebook haben wir eine Multi-Klassen-Objekterkennung implementiert, die sowohl Personen als auch Autos in Bildern erkennen kann. Hier sind die wichtigsten Punkte:

1. **Training eines Personenerkennungsmodells:** Wir haben ein CNN-Modell trainiert, um Personen im CIFAR-10-Datensatz zu erkennen, ähnlich wie wir es für die Automerkennung getan haben.
2. **Kombination von Modellen:** Wir haben das Personenerkennungsmodell mit dem vor-trainierten Autoerkennungsmodell kombiniert, um beide Objektklassen in Bildern zu erkennen.
3. **Multi-Klassen-Objekterkennung:** Wir haben einen Algorithmus implementiert, der Region Proposals generiert, beide Modelle auf jede Region anwendet und Non-Maximum Suppression für jede Klasse durchführt.
4. **Visualisierung der Ergebnisse:** Wir haben die Erkennungsergebnisse visualisiert, wobei verschiedene Farben für verschiedene Objektklassen verwendet wurden.
5. **Parameteroptimierung:** Wir haben verschiedene Parameter wie den Konfidenz-Schwellenwert und den IoU-Schwellenwert getestet, um die Leistung des Algorithmus zu optimieren.

Diese Implementierung zeigt, wie man spezialisierte Modelle für verschiedene Objektklassen kombinieren kann, um komplexere Aufgaben zu lösen. Der modulare Ansatz ermöglicht es, neue Objektklassen hinzuzufügen, ohne bestehende Modelle neu trainieren zu müssen.

Für zukünftige Verbesserungen könnten wir: - Weitere Objektklassen hinzufügen (z.B. Fahrräder, Verkehrsschilder) - Fortgeschrittenere Modelle wie YOLO oder Faster R-CNN verwenden - Die Effizienz des Algorithmus verbessern, um Echtzeiterkennung zu ermöglichen - Die Robustheit gegenüber verschiedenen Lichtbedingungen, Perspektiven und Verdeckungen verbessern

Insgesamt bietet diese Implementierung eine gute Grundlage für die Entwicklung komplexerer Objekterkennungssysteme und zeigt die Flexibilität und Erweiterbarkeit von CNN-basierten Ansätzen.